
Manage Your Plug-in Data for Language Models: A Data-Centric Approach

Jiarui Jin^{1,†,‡}, Yuwei Wu^{2,†}, Mengyue Yang^{3,†}, Xiaoting He⁴,
Weinan Zhang¹, Yiming Yang², Yong Yu¹, Jun Wang³
¹Shanghai Jiao Tong University, ²Carnegie Mellon University,
³University College London, ⁴No Affiliation

Abstract

Large language models (LMs) achieve remarkable few-shot performance by simply prompting LMs with a few demonstrations as the input context, without the need for computationally expensive fine-tuning. Here, the core question is *what kind of demonstrations are highly effective and how to find them efficiently?* Our insight is that the effectiveness of demonstrations would be conditioned on the LMs used. Unlike previous approaches that select similar and/or diverse demonstrations regardless of LMs, we propose GUNDAM (Golden PLUG-IN DATA Manager), a novel framework that measures *sufficiency* and *necessity* of plugging-in demonstrations conditioned on LMs. We show that the proposed sufficiency and necessity metrics can be operated on both demonstration instances (i.e., instance level) and demonstration sets (i.e., set level). Thus, the set of sufficient and necessary plug-in data, named as golden plug-in set, can be regarded as a core set including the informative training samples. Since enumerating and measuring all the possible subsets is infeasible, we design new tree-based search algorithms to mine the golden plug-in set. Notice that the golden plug-in set could be pre-computed and stored to save online computation costs. In this regard, non-parametric demonstration retrievers could perform on the stored golden plug-in set instead of the training dataset, to avoid retrieving insufficient or unnecessary demonstrations. Moreover, considering that the training dataset would keep growing in many real-world scenarios, we develop an incremental update algorithm to avoid the re-computing GUNDAM over all the changed and unchanged parts. We also show that GUNDAM can be straightforwardly extended to the supervised learning setting by organizing fine-tuning LMs into a unified bi-level optimization paradigm. Our empirical results on 6 datasets show that GUNDAM can consistently achieve a better (or comparable) performance of three GPT variants in 1-shot, 2-shot, 5-shot, and 10-shot settings with three different retrievers, meanwhile saving half the size of the input training data.

1 Introduction

Language models (LMs) especially large LMs such as T5 [27], GPT-3 [2] and CodeX [4] have shown impressive performance on a wide range of downstream tasks by simply prompting a few examples (termed as demonstrations). This type of few-shot inference, known as in-context learning [2, 6], can avoid the high computation costs of fine-tuning LMs. The core challenge is how to select the most effective demonstrations from all the possible candidates in a large training set. Most of the early methods select the demonstrations based on relevance, using the similarity score of each

[†]Equal contributions: Jiarui, Yuwei, and Mengyue.

[‡]Work done during Jiarui’s visit at University College London.

demonstration with respect to the input question. Previous studies [25, 19, 29, 34] select those training examples similar to the input question. As pointed out in a recent work [17], measuring each example in isolation would lead to a sub-optimal solution. Although sampling based on diversity in addition to similarity is a better remedy, this idea in fact coincides with the well-know principle named maximum marginal relevance (MMR) [3] for unsupervised query expansion in information retrieval. We argue that all these previous metrics should be re-investigated in depth in the new era of large LMs, since they measure each instance regardless of LMs.

In this paper, our main idea is that the effectiveness of demonstrations would be conditioned on the LMs used. To this end, we propose, a new framework, named GUNDAM (Golden PlUG-in DAta Miner), to answer *what kind of demonstrations are highly effective and how to find them efficiently*.

We begin by investigating *sufficiency* and *necessity* of plugging-in (a.k.a., prompting) each demonstration, which establishes a new principle in settings for optimizing the performance of LMs. Specifically, sufficiency measures whether the plug-in demonstration would improve the performance of LMs on domain-specific tasks, while necessity measures a newly selected plug-in demonstration is redundant to those already selected before. To measure the quality of selected plug-in data as a whole, we then extend the sufficiency and necessity metrics from the instance level (i.e., measuring whether plugging-in each data point is sufficient and necessary in terms of inferring other data) to the set level (i.e., measuring whether plugging-in each set of data is sufficient and necessary in terms of inferring other sets of data). We call the selected subset (from the training dataset) of sufficient and necessary demonstrations as a *golden plug-in set*.

To extract a golden plug-in set from the training dataset, enumerating and evaluating each possible plug-in set is super time-consuming. Therefore, we develop tree-based algorithms to compute the golden plug-in set in an efficient miner. In this regard, our approach is an unsupervised core-set selection method that produces an informative subset of training instances. Hence, for each training dataset, we could pre-compute and store the golden plug-in set to save the computations for the downstream tasks. Then, GUNDAM can collaborate with various demonstration retrievers by using the golden plug-in set as the retrieval pool instead of the whole training dataset, to scale GUNDAM up to handle large-scale data. Since the training set would keep growing larger and larger in many real-world scenarios (e.g., knowledge database), we further develop an incremental algorithm to allow only running GUNDAM over those changed examples (including newly added and modified ones) instead of re-computing over all the changed and unchanged training examples.

We also show that it is straightforward to extend GUNDAM from the unsupervised learning setting to the supervised learning setting for LMs. The process of fine-tuning LM framework can be organized as first retrieving high-quality data from reference datasets and then using them as extra input to LMs when needed, to reduce hallucination and increase coverage [16, 1, 15, 37]. To this end, we formulate the fine-tuning problem into a unified bi-level optimization framework. It consists of an outer loop to extract high-quality data with a *frozen LM*, and an inner loop to optimize LMs with *fixed plug-in data*, where one can directly apply GUNDAM in the outer loop.

Our empirical results on 6 datasets illustrate that GUNDAM can efficiently select golden plug-in data from the whole training datasets, saving almost half the size. Meanwhile, using the selected subset instead of the training dataset can achieve a better or comparable performance in 1-shot, 2-shot, 5-shot, and 10-shot settings, by collaborating it with a random or a similarity-based or a diversity-based retriever, on the basis of three GPT variants.

2 A Data-centric View for In-Context Learning and Fine-tuning

We begin by introducing GUNDAM within two different contexts: the in-context learning (a.k.a., unsupervised learning) setting and the fine-tuning (a.k.a., supervised learning) setting. We explore both of them from a data-centric viewpoint [32], where *data quality* outweighs *data quantities*.

Unsupervised Learning Setting. Given a training dataset $\mathcal{D}_{\text{TRAIN}} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$ containing pairs of input data (e.g., questions) and output labels (i.e., answers); and a test dataset $\mathcal{D}_{\text{TEST}} = \{(\mathbf{x}_m, \mathbf{y}_m)\}_{m=1}^M$, where we assume that $\mathcal{D}_{\text{TRAIN}}$ is a support set [39] of $\mathcal{D}_{\text{TEST}}$. Our goal is to design a demonstration miner to mine a subset of training examples $\tilde{\mathcal{D}}_{\text{TRAIN}} \subset \mathcal{D}_{\text{TRAIN}}$ where $\tilde{\mathcal{D}}_{\text{TRAIN}} = \{(\mathbf{x}_p, \mathbf{y}_p)\}_{p=1}^P$ and $P \ll N$. We use $\Psi_{\text{LM}} : \mathbb{X} \times \mathbb{D} \rightarrow \mathbb{Y}$ to denote a retrieval-augmented LM. Here, $\mathbf{x} \in \mathbb{X}$ is an input question, $\mathbf{y} \in \mathbb{Y}$ is the corresponding answer, and $(\mathbf{x}, \mathbf{y}) \in \mathbb{D}$ is a plug-in

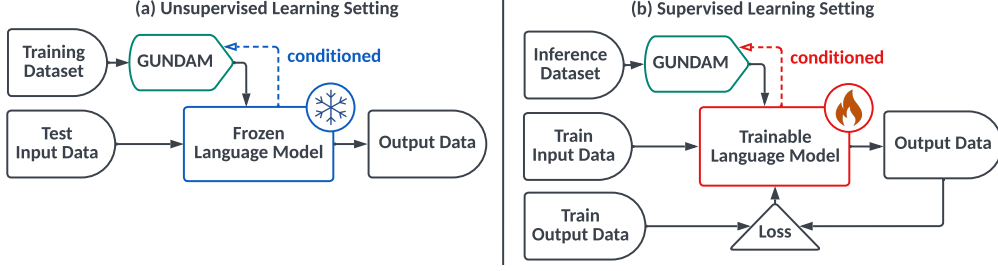


Figure 1: Overview of GUNDAM. In the unsupervised learning setting, shown in (a), GUNDAM is designed to select a sufficient and necessary plug-in set from the training dataset as the input demonstrations, where sufficiency and necessity metrics are conditioned on the frozen LM. In the supervised learning setting, as shown in (b), GUNDAM can be applied to mine a high-quality set from the reference dataset, where sufficiency and necessity metrics are conditioned on the trainable LM.

demonstration. The LM $\Psi_{\text{LM}}(\cdot)$ takes both the input questions and selected demonstrations as the input and outputs the predicted answers. Formally, our goal is to minimize:

$$\mathcal{L}(\tilde{\mathcal{D}}_{\text{TRAIN}}, \mathcal{D}_{\text{TEST}}; \Psi_{\text{LM}}^*) = \sum_{(\mathbf{x}_m, \mathbf{y}_m) \in \mathcal{D}_{\text{TEST}}} \ell(\Psi_{\text{LM}}^*(\mathbf{x}_m, \tilde{\mathcal{D}}_{\text{TRAIN}}), \mathbf{y}_m), \quad (1)$$

where $\ell(\cdot)$ is the given task-specific loss function, and $\Psi_{\text{LM}}^*(\cdot)$ means that the LM is frozen. Since we can not get access to $\mathcal{D}_{\text{TEST}}$, it is impractical to optimize the miner (i.e., $\tilde{\mathcal{D}}_{\text{TRAIN}}$) via directly minimizing $\mathcal{L}(\tilde{\mathcal{D}}_{\text{TRAIN}}, \mathcal{D}_{\text{TEST}})$.

Instead, we consider the demonstration selection as selecting high-quality demonstrations from the training samples. Our main idea is that high-quality training examples (i.e., $\tilde{\mathcal{D}}_{\text{TRAIN}}$) should be representative of all the data points in $\mathcal{D}_{\text{TRAIN}}$, meanwhile, its size is expected to be minimal as possible. Formally, we formulate our objective as:

$$\min_{\tilde{\mathcal{D}}_{\text{TRAIN}} \subset \mathcal{D}_{\text{TRAIN}}} |\tilde{\mathcal{D}}_{\text{TRAIN}}|, \text{ s.t. } \mathcal{L}(\tilde{\mathcal{D}}_{\text{TRAIN}}, \mathcal{D}_{\text{TRAIN}}; \Psi_{\text{LM}}^*) \geq \mathcal{L}(\mathcal{D}_{\text{TRAIN}}, \mathcal{D}_{\text{TRAIN}}; \Psi_{\text{LM}}^*). \quad (2)$$

It requires that $\tilde{\mathcal{D}}_{\text{TRAIN}}$ should be both *sufficient* and *necessary* to represent $\mathcal{D}_{\text{TRAIN}}$, namely removing redundant data points to save computation costs, meanwhile maintaining the performance of the LM.

Supervised Learning Setting. The above data-centric viewpoint of mining the high-quality set also works for fine-tuning LMs. Given a training dataset $\mathcal{D}_{\text{TRAIN}} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$, a test dataset $\mathcal{D}_{\text{TEST}} = \{(\mathbf{x}_m, \mathbf{y}_m)\}_{m=1}^M$, and a reference dataset $\mathcal{D}_{\text{REFER}} = \{(\mathbf{x}_q, \mathbf{y}_q)\}_{q=1}^Q$. Our goal to minimize $\mathcal{L}(\tilde{\mathcal{D}}_{\text{REFER}}, \mathcal{D}_{\text{TEST}})$. Here, $\tilde{\mathcal{D}}_{\text{REFER}}$ is a set of retrieved data points from $\mathcal{D}_{\text{REFER}}$. Different from the above unsupervised learning setting, the LM $\Psi_{\text{LM}}(\cdot)$ is usually trainable.

We formulate the above problem into a bi-level optimization framework regarding $\mathcal{L}(\tilde{\mathcal{D}}_{\text{REFER}}, \mathcal{D}_{\text{TRAIN}})$, since $\mathcal{D}_{\text{TRAIN}}$ is assumed as a support set of $\mathcal{D}_{\text{TEST}}$. In the outer loop, we optimize the miner to extract $\tilde{\mathcal{D}}_{\text{REFER}}$ in the context of the frozen LM. Formally, our objective is:

$$\min_{\tilde{\mathcal{D}}_{\text{REFER}} \subset \mathcal{D}_{\text{REFER}}} |\tilde{\mathcal{D}}_{\text{REFER}}|, \text{ s.t. } \mathcal{L}(\tilde{\mathcal{D}}_{\text{REFER}}, \mathcal{D}_{\text{TRAIN}}; \Psi_{\text{LM}}^*) \geq \mathcal{L}(\mathcal{D}_{\text{REFER}}, \mathcal{D}_{\text{TRAIN}}; \Psi_{\text{LM}}^*). \quad (3)$$

In the inner loop, we optimize the LM with the fixed retrieved reference data, where the objective is:

$$\min_{\Psi_{\text{LM}}} \mathbb{E}_{(\mathbf{x}_n, \mathbf{y}_n) \in \mathcal{D}_{\text{TRAIN}}} [\ell(\Psi_{\text{LM}}(\mathbf{x}_n, \tilde{\mathcal{D}}_{\text{REFER}}^*), \mathbf{y}_n)], \quad (4)$$

where $\tilde{\mathcal{D}}_{\text{REFER}}^*$ means that the reference plug-in data is fixed.

Connections to Related Work. With the increasing capability of large LMs, in-context learning [6] has become a new paradigm for the natural language process, where the central issue is how to select training examples (a.k.a., demonstrations). Previous solutions lie in building either a parameter-free retriever [34, 40] or a neural-based retriever [22, 19, 14, 29]. As pointed out in [17], greedily retrieving demonstrations similar to the input question could lead to sub-optimal performance.

Different from the above retriever operating on the instance level (i.e., separately measuring the similarity of each instance), GUNDAM is developed based on the sufficiency and necessity metrics and

is executed at the set level (i.e., comprehensively considering the sufficiency and necessity of each set instead of each data point).

We note that our approach can be regarded as an unsupervised learning core-set selection method, and we summarize the connections to other existing methods in Appendix A3.

3 GUNDAM: Mining Sufficient and Necessary Plug-in Data

Let X, Z denote variables for the input question and the (previous) plugged-in data point. We use Y , a boolean variable, to denote whether the answer to the input question is right. For simplicity, we use $Y_{x_n} = 1$ to denote $Y = 1|X = x_n$, meaning that the LM generates the right answer to the input question x_n . Similarly, $Y_{x_n} = 0$, equivalent to $Y = 0|X = x_n$, indicates that the LM produces the wrong answer to x_n . For clarity, we introduce S , a variable to record the original status of the LM before *new* plug-in and unplug operations (denoted as $\text{plug}(\cdot)$ and $\text{unplug}(\cdot)$ respectively), e.g., $Z = ((x_n, y_n))$, $S = (Y_{x_n} = 1)$ means that without plugging-in any new data or unplugging any plugged-in data, the plugged-in data is (x_n, y_n) and the LM’s performance is $Y_{x_n} = 1$.

Assumption. We assume that for any data point (x_n, y_n) , plugging-in itself can ensure that the LM’s answer to x_n is right. Namely, $Y_{x_n} = 1|\text{plug}((x_n, y_n)); Z, S$ always holds for any values assigned to Z and S .

In the following subsections, we first introduce GUNDAM in the context of the unsupervised learning setting, from theory (in Section 3.1) to practice (in Sections 3.2 and 3.3). As GUNDAM can be directly in the outer loop of the supervised learning setting, we leave detailed descriptions and discussions in Appendix A2.4.

3.1 Relationship between Plug-in Data and Outputs: From Instance level to Set level

We begin by investigating the sufficiency and necessity of plugging each training example (x_n, y_n) regarding target data point (x_m, y_m) in the instance level.

Definition 1 (Sufficient Plug-in Instance). Given tuple (X, Y, Z, S) , we say that a training example (x_n, y_n) is a sufficient plug-in instance for a target data point (x_m, y_m) (namely plugging-in (x_n, y_n) is sufficient to get the right answer of x_m), if the following equation holds:

$$Y_{x_m} = 1|\text{plug}((x_n, y_n)); Z = \emptyset, S = (Y_{x_m} = 0). \quad (5)$$

It indicates that before plugging-in (x_n, y_n) , there is no plugged-in data (i.e., $Z = \emptyset$) and the LM’s answer to x_m is wrong (i.e., $S = (Y_{x_m} = 0)$), and when we plug-in (x_n, y_n) (i.e., $\text{plug}((x_n, y_n))$), then it would correct the LM’s answer to x_m (i.e., $Y_{x_m} = 1$).

Definition 2 (Necessary Plug-in Instance). Given tuple (X, Y, Z, S) , we say that a training example (x_n, y_n) is a necessary plug-in instance for a target data point (x_m, y_m) (namely plugging-in (x_n, y_n) is necessary to get the right answer of x_m), if the following equation holds:

$$Y_{x_m} = 0|\text{unplug}((x_n, y_n)); Z = ((x_n, y_n)), S = (Y_{x_m} = 1). \quad (6)$$

It means that before unplugging (x_n, y_n) , there is plugged-in data (x_n, y_n) (i.e., $Z = ((x_n, y_n))$) and the LM’s answer to x_m is right (i.e., $S = (Y_{x_m} = 1)$), and when we unplug (x_n, y_n) (i.e., $\text{unplug}((x_n, y_n))$), then it would cause the LM’s answer to x_m being wrong (i.e., $Y_{x_m} = 0$).

We illustrate the above instance-level sufficiency and necessity metrics by providing some examples in Appendix A1.1. Combining Definitions 1 and 2, it is easy to reach the definition of sufficient and necessary plug-in instance. To save the space, we give the detailed definition in Definition A3 and provide Lemma A1 to show that sufficiency and necessity are not two independent properties in Appendix A1.1. Based on the above instance-level definitions, we can extend them to the set level, where sufficient plug-in set indicates that plugging-in one set is sufficient to get the right answer of any data point in a target set, while necessary plug-in set means that unplugging any data point in one set would cause the answers to at least one data points in a target set from right to wrong. We provide formal set-level definitions and some examples in Appendix A1.2.

Then, we consider the high-quality set of the training examples as a subset of $\mathcal{D}_{\text{TRAIN}}$, denoted as $\tilde{\mathcal{D}}_{\text{TRAIN}}$, that is both sufficient and necessary to represent the whole training set, namely plugging-in $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is sufficient and necessary for the LM to infer $\mathcal{D}_{\text{TRAIN}}$. We name this subset as a golden plug-in set for $\mathcal{D}_{\text{TRAIN}}$. Formally, we introduce its definition as follows.

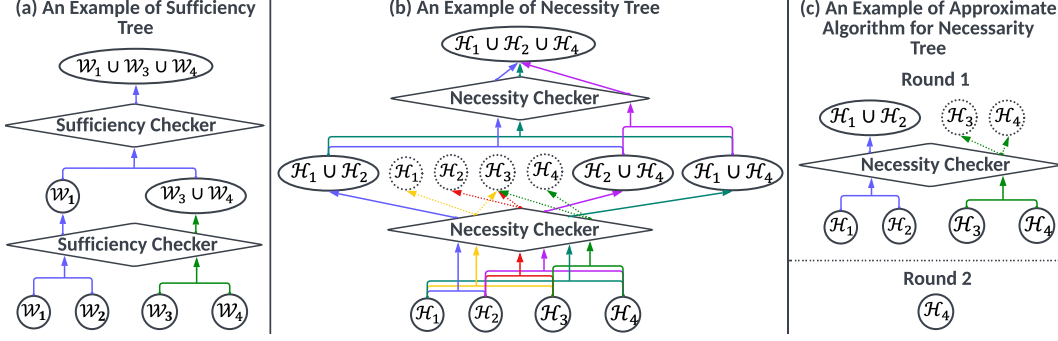


Figure 2: An illustrated example of mining sufficient and necessary plug-in set through building sufficiency and necessity trees. In (a), we use a sufficiency checker to check each pair of nodes, and after checking, we remove them from $\mathcal{W}$; in other words, each node would only be checked once. In (b), we run a necessity checker to check each pair of nodes, and we do not remove them from $\mathcal{H}$; instead, we assign MAINTAIN signals to newly generated nodes and the node with the maximum size, and those nodes without MAINTAIN signals, circled with dashed lines, would be removed from $\mathcal{H}$. In (c), we approximate (b) by removing nodes after checking (similar to (a)), and we repeat the above process for multiple rounds, and at the beginning of each round, we unplug all the previously selected data points. The repeat should stop until there is no or only one node in $\mathcal{H}_0$ (i.e., $\mathcal{H}_4$), and therefore, the result in (c) is $\mathcal{H}_1 \cup \mathcal{H}_2 \cup \mathcal{H}_4$, same as the result in (b).

Definition 3 (Golden Plug-in Set). Given tuple (X, Y, Z) , according the assumption posted in the beginning of Section 3, we derive that $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N | \text{plug}(\mathcal{D}_{\text{TRAIN}}); Z = \emptyset$ holds. Then, we say that $\mathcal{D}_{\text{GOLD}}$, a subset of $\mathcal{D}_{\text{TRAIN}}$, is a golden plug-in set, if the following conditions are satisfied:

- (i) $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N | \text{plug}(\mathcal{D}_{\text{GOLD}}); Z = \emptyset$ holds, where $\mathbf{1}_N$ and $\mathbf{0}_N$ denotes N -dimensional vectors whose elements are all 1s and 0s. It indicates that plugging-in $\mathcal{D}_{\text{GOLD}}$ alone is sufficient for maintaining $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N$.
- (ii) $Y_{(x_1, \dots, x_N)} \neq \mathbf{1}_N | \text{unplug}(\mathcal{D}'_{\text{GOLD}} \cup \overline{\mathcal{D}_{\text{GOLD}}}); Z = \mathcal{D}_{\text{TRAIN}}$ holds for any subset of $\mathcal{D}_{\text{GOLD}}$ (denoted as $\mathcal{D}'_{\text{GOLD}}$), where $\overline{\mathcal{D}_{\text{GOLD}}} = \mathcal{D}_{\text{TRAIN}} - \mathcal{D}_{\text{GOLD}}$. It indicates that if $\overline{\mathcal{D}_{\text{GOLD}}}$ would be unplugged, then unplugging any data point in $\mathcal{D}_{\text{GOLD}}$ would make the plugged-in data not sufficient to keep the answers right for all the input questions. Namely, plugging-in $\mathcal{D}_{\text{GOLD}}$ is necessary for maintaining $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N$.

We demonstrate the golden plug-in set with some examples, and draw connections between the golden plug-in set and sufficient and necessary plug-in set in in Appendix A1.2.

3.2 Mining Golden Plug-in Data with Tree Search

To mine $\mathcal{D}_{\text{GOLD}}$ from $\mathcal{D}_{\text{TRAIN}}$, one straightforward approach is to enumerate and evaluate all the possible subsets of $\mathcal{D}_{\text{TRAIN}}$. However, it is infeasible in practice, since it would require $O(2^N)$ computations. To address this, we develop two tree-search algorithms that can separately extract sufficient plug-in set and necessary plug-in set from the given dataset. In the following, we respectively describe these two algorithms, and for convenience, we use $\mathcal{D}_{\text{IN}} = \{(x_n, y_n)\}_{n=1}^{N_{\text{IN}}}$ to denote the input dataset for these two algorithms, and use $\mathcal{D}_{\text{OUT}}$ to denote the output set. Both two trees grow from bottom to top. Let K denote the depth of these trees (i.e., the number of iterations), and we use $k = 1, \dots, K$ to denote the k -th iteration to produce the $(k+1)$ -th layer of the tree.

Sufficiency Tree Search. Noticing that LMs (especially large LMs) are capable of inferring with a chain of thoughts [36], we assume that sufficiency is transitive among set, namely if $\mathcal{D}_A$ is a sufficient plug-in set of $\mathcal{D}_B$, and $\mathcal{D}_B$ is a sufficient plug-in set of $\mathcal{D}_C$, then $\mathcal{D}_A$ is a sufficient plug-in set of $\mathcal{D}_C$. Here, we leverage the transitivity of sufficiency to build a sufficiency tree, where each node is a set of data points. Formally, we use $\mathcal{W}_k$ to denote a set of nodes after the k -th iteration. We initialize $\mathcal{W}_0$ by assigning all the training data points as the bottom nodes:

$$\mathcal{W}_0 := \{\mathcal{W}_n := \{(x_n, y_n)\} | (x_n, y_n) \in \mathcal{D}_{\text{IN}}\}. \quad (7)$$

At each k -th iteration, we introduce a *sufficiency checker* to produce $\mathcal{W}_k$ by studying the sufficiency relationship between each pair of nodes (denoted as $\mathcal{W}_i, \mathcal{W}_j \in \mathcal{W}_{k-1}$). Here, we check whether the

following equation holds by assigning $\mathcal{W}_i$ and $\mathcal{W}_j$ as $\mathcal{W}_{\text{IN}}$ and $\mathcal{W}_{\text{OUT}}$, or, $\mathcal{W}_{\text{OUT}}$ and $\mathcal{W}_{\text{IN}}$, respectively:

$$Y_{(\{x_n | x_n \in \mathcal{W}_{\text{OUT}}\})} = \mathbf{1}_W | \text{plug}(\mathcal{W}_{\text{IN}}); Z = \emptyset, \quad (8)$$

where $W = |\mathcal{W}_{\text{OUT}}|$. If Eq. (8) holds, it indicates that plugging $\mathcal{W}_{\text{IN}}$ is sufficient for the LM to get the right answer to any input question in $\mathcal{W}_{\text{OUT}}$. Then, there are three possible cases: (i) Both $\mathcal{W}_i$ and $\mathcal{W}_j$ are sufficient plug-in set for each other, then we append the one with fewer elements to $\mathcal{W}_k$; (ii) If only one of $\mathcal{W}_i$ and $\mathcal{W}_j$ is a sufficient plug-in set for the other one, then we append the sufficient plug-in set to $\mathcal{W}_k$; (iii) If neither $\mathcal{W}_i$ nor $\mathcal{W}_j$ is a sufficient plug-in set, then we append $\mathcal{W}_i \cup \mathcal{W}_j$ to $\mathcal{W}_k$. After the above computation for each pair of nodes, we should remove them from $\mathcal{W}_{k-1}$, and once there is only one element left in $\mathcal{W}_{k-1}$, we should directly append it to $\mathcal{W}_k$. The above process would loop until $\mathcal{W}$ only includes one element (i.e., the root node), denoted as $\mathcal{W}_{\text{SUFFICIENT}} \in \mathcal{W}_K$. We then compute $\mathcal{D}_{\text{OUT}} = \mathcal{W}_{\text{SUFFICIENT}}$.

We provide an illustrated example of the above process in Figure 2(a), and running the sufficiency tree search algorithm requires $O(\log_2^{N_{\text{IN}}})$ iterations.

Necessity Tree Search. Notice that if unplugging $\mathcal{D}_A$ could cause the answers to at least one input question in $\mathcal{D}_C$ from right to wrong, then unplugging $\mathcal{D}_A \cup \mathcal{D}_B$ also can not maintain the answers to all the data points in $\mathcal{D}_C$ right. Here, we aim to extract the unnecessary part from $\mathcal{D}_{\text{IN}}$ to make the rest part a necessary plug-in set. To this end, we build a necessity tree, where each node includes a set of unnecessary data points. Formally, we use $\mathcal{H}_k$ to denote a set of nodes after the k -th iteration. We initialize $\mathcal{H}_0$ by summarizing all the data points in $\mathcal{D}_{\text{IN}}$ satisfying: solely unplugging each of them does not influence the LM's performance. Formally, we construct $\mathcal{H}_0 := \{\mathcal{H}_n := \{(x_n, y_n)\}\}$ where $(x_n, y_n) \in \mathcal{D}_{\text{IN}}$ satisfies:

$$Y_{(\{x_{n'} | x_{n'} \in \mathcal{D}_{\text{IN}}\})} = \mathbf{1}_{N_{\text{IN}}} | \text{unplug}((x_n, y_n)); Z = \mathcal{D}_{\text{IN}}. \quad (9)$$

At each k -th iteration, we introduce a *necessity checker* to produce $\mathcal{H}_k$ by studying the necessary relationship between each pair of nodes (denoted as $\mathcal{H}_i, \mathcal{H}_j \in \mathcal{H}_{k-1}$). Here, we check whether solely unplugging $\mathcal{H}_i \cup \mathcal{H}_j$ does not influence the LM's performance, formally, whether the following equation holds:

$$Y_{(\{x_{n'} | x_{n'} \in \mathcal{D}_{\text{IN}}\})} = \mathbf{1}_{N_{\text{IN}}} | \text{unplug}(\mathcal{H}_i \cup \mathcal{H}_j); Z = \mathcal{D}_{\text{IN}}, \quad (10)$$

which indicates that plugging $\mathcal{H}_i \cup \mathcal{H}_j$ is unnecessary for maintaining all the answers to the input questions in $\mathcal{D}_{\text{IN}}$ right. Then, if the above equation holds, we generate a new node $\mathcal{H}_i \cup \mathcal{H}_j$, and append it to $\mathcal{H}_k$, and assign $\mathcal{H}_i \cup \mathcal{H}_j$ with MAINTAIN signals. Otherwise, we append $\mathcal{H}_i$ and $\mathcal{H}_j$ to $\mathcal{H}_k$. After the above computation, we compute $\mathcal{H}_{\text{MAX}} = \arg \max_{\mathcal{H}_i \in \mathcal{H}_k} |\mathcal{H}_i|$ and assign it with a MAINTAIN signal. Then, we remove the nodes without MAINTAIN signals in $\mathcal{H}_k$. The above process should repeat until $\mathcal{H}$ only includes one element (i.e., the root node), denoted as $\mathcal{H}_{\text{UNNECESSARY}} \in \mathcal{H}_K$. We then compute $\mathcal{D}_{\text{OUT}} = \mathcal{D}_{\text{IN}} - \mathcal{H}_{\text{UNNECESSARY}}$.

We provide an illustrated example of the above process in Figure 2(b), and since at each iteration, we need to run the checker for $O(\mathcal{C}_{N_{\text{IN}}}^2)$ times (where $\mathcal{C}$ denotes a combination operator), which is intolerable in practice. Therefore, we develop an alternative algorithm. Concretely, at each k -th iteration, we remove the checked nodes (i.e., $\mathcal{H}_i$ and $\mathcal{H}_j$) from $\mathcal{H}_k$ (similar to the sufficiency tree search). Then, similarly, it requires $O(\log_2^{N_{\text{IN}}})$ iterations to finish one round. To obtain a necessary plug-in set, we need to repeat the above process for multiple rounds, where at the beginning of each round, we unplug all the selected data points in the previous rounds. The repeat should stop until there is no or only one node in $\mathcal{H}_0$. When practical, we can set R_{MAX} as the maximum number of rounds to approximate, and overall this approximate algorithm requires $O(R_{\text{MAX}} \log_2^{N_{\text{IN}}})$ iterations.

Theorem 1. *If we successively apply the sufficiency tree search algorithm and the necessity tree search algorithm (either the original one or the alternative one) on $\mathcal{D}_{\text{TRAIN}}$ to obtain $\tilde{\mathcal{D}}_{\text{TRAIN}}$, then $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a golden plug-in set of $\mathcal{D}_{\text{TRAIN}}$.*

Lemma 1. *If we directly apply the necessity tree search algorithm (either the original one or the alternative one) on $\mathcal{D}_{\text{TRAIN}}$ to obtain $\tilde{\mathcal{D}}_{\text{TRAIN}}$, then $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a golden plug-in set of $\mathcal{D}_{\text{TRAIN}}$.*

Combining Theorem 1 and Lemma 1, we can see that the sufficiency tree search algorithm can be used as an efficient filter to remove the insufficient part from $\mathcal{D}_{\text{TRAIN}}$. Therefore, we can recursively apply the sufficiency tree search algorithm for multiple rounds, to obtain a condensed sufficient plug-in set. We provide detailed descriptions of the above tree-search algorithms in Algorithms A1, A2 and A3, together with the analysis and proof for the above theorem and lemma in Appendix A2.1.

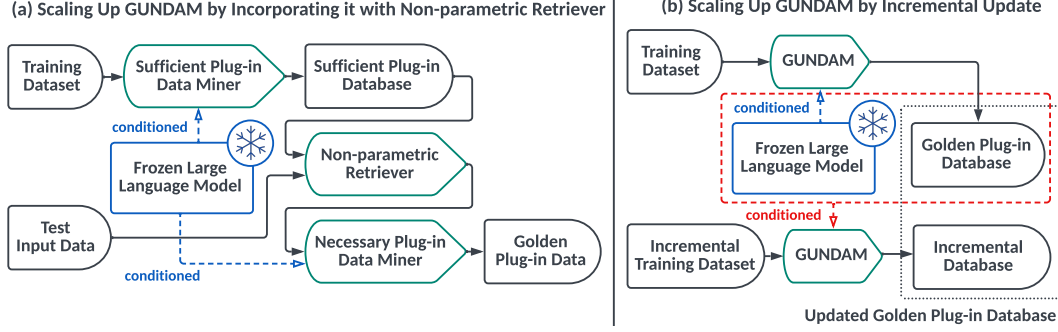


Figure 3: Scaling up GUNDAM to real-world applications, where (a) we first run the sufficiency tree algorithm, then run a retriever to retrieve items according to test data, and run the necessity tree algorithm to remove the unnecessary part from the retrieved data; (b) we develop an incremental update algorithm to avoid re-computing over all the unchanged and changed training examples, which can both used to re-compute the sufficiency and necessary plug-in set of the given datasets, and used to incrementally select each demonstration.

3.3 Scaling Up GUNDAM into Real-World Applications

In the above subsections, we have introduced what is golden plug-in set and how to mine it from the given data. However, there are two limitations of GUNDAM in real-world applications.

Scaling Up GUNDAM by incorporating it with Non-parametric Retriever. One limitation is that the produced $\hat{\mathcal{D}}_{\text{TRAIN}}$ is too large to be directly used as input demonstrations. For this purpose, we incorporate GUNDAM to existing demonstration retrievers to retrieve relevant demonstrations from $\hat{\mathcal{D}}_{\text{TRAIN}}$, namely *first mining the golden plug-in set, and then retrieval*. In this regard, our approach can be used as an unsupervised learning core-set selection method that mines informative training samples from the whole training dataset, to benefit the downstream tasks. However, directly applying the necessity search algorithm on sufficiency-filtered data could also be problematic, since $\mathcal{D}_{\text{IN}}$ would be too large to be prompted. Therefore, we design to place the retriever between the sufficient plug-in data miner and the necessary plug-in data miner. Then, we only need to filter the unnecessary part from the retrieved demonstrations. We depict the above resulting process in Figure 3(a). Here, the input is the whole training dataset, and the output is the golden plug-in set, and thus the whole process corresponds to GUNDAM component in Figure 1. We note that we could run the sufficiency tree search *multiple rounds* to obtain a condensed sufficient plug-in set before employing the retriever.

Scaling Up GUNDAM by Incremental Update. Another limitation is that many real-world (training) datasets are temporal (and some even require daily updates), and directly re-calculating the golden plug-in set over all the unchanged and changed samples is time-consuming. To address this, we develop a new incremental update algorithm for GUNDAM allowing us to only re-compute the changed parts (including new adding and modified samples). As shown in Figure 3(b), once we generate the golden plug-in set for the original dataset, then we can treat the unchanged part of plug-and-play plug-in data and the LM as the whole (shown as the dashed box), as a new “LM model”, and therefore, we only apply GUNDAM to compute incremental plug-in data for the changed part (including newly added and modified data points). When incorporating into the above process shown in Figure 3(a), we could store and update the sufficient plug-in database instead of the golden plug-in database, and treat the retrieved data, instead of the whole plug-in data, and the LM as a whole.

We summarize the overall algorithm of scaling up GUNDAM in unsupervised learning setting and supervised learning setting in Algorithms A4 and A5 respectively, together with the corresponding analysis in Appendix A2.3 and A2.4.

4 Experiment

4.1 Experimental Configuration

We use six text classification datasets, i.e., SST-2 [30], SST-5 [30], COLA [35], TREC [33], SUBJ [21], and FPB [20], ranging from sentiment classification, linguistic analysis to textual entailment. We use two GPT-2 variants [26] (with 335M parameters (denoted as MED), 774M parameters (denoted as LAR), and one GPT-3 1.3B variant (denoted as NEO) as the LM (i.e., $\Psi_{\text{LM}}(\cdot)$). Due to the limited

Table 1: Result comparisons on text classification datasets for different LM bases (MED, LAR, NEO) with different retrievers (RAN, SIM, DIV) in different shot settings (1, 2, 5, 10). All the results are calculated based on 5 different random seeds, and the bold numbers mean using $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{TRAIN}}$ can obtain at least 2% improvements than using $\mathcal{D}_{\text{TRAIN}}$, or vice versa. See Table A1 for more extended results (on datasets SUBJ, and FPB).

$\Psi_{\text{LM}}(\cdot)$	$\tilde{\mathcal{D}}_{\text{TRAIN}}$	n	SST-2			SST-5			COLA			TREC		
			RAN	SIM	DIV	RAN	SIM	DIV	RAN	SIM	DIV	RAN	SIM	DIV
MED	$\mathcal{D}_{\text{TRAIN}}$	1	48.9	24.5	24.5	14.5	22.7	22.7	29.0	38.8	38.8	19.4	42.8	42.8
		2	51.2	65.7	62.5	18.0	25.6	23.7	30.9	38.5	36.2	21.4	57.2	51.4
		5	62.6	79.4	61.7	26.5	32.3	27.8	69.4	49.3	47.0	37.6	66.0	61.4
		10	50.9	83.8	76.9	14.9	35.3	30.4	31.6	52.5	58.8	53.0	71.4	65.8
	$\mathcal{D}_{\text{GOLD}}$	1	49.8	48.1	48.1	15.4	23.7	23.7	29.4	35.1	35.1	37.4	48.4	48.4
		2	67.3	67.7	64.7	20.9	27.9	25.8	31.1	41.7	34.9	27.6	56.8	52.2
		5	70.3	77.9	68.5	28.6	33.2	27.4	65.2	57.3	54.6	40.8	67.4	61.8
		10	75.2	83.0	77.2	17.6	36.2	29.8	69.3	68.7	68.5	44.6	74.6	64.6
LAR	$\mathcal{D}_{\text{TRAIN}}$	1	49.0	42.3	42.3	14.2	25.2	25.2	42.1	38.3	38.3	21.0	53.2	53.2
		2	68.0	70.7	59.6	18.1	29.7	24.4	41.1	36.8	37.7	28.2	62.6	60.6
		5	49.1	80.6	67.5	25.6	34.1	30.8	66.2	53.8	48.5	35.4	63.4	64.6
		10	71.1	84.6	73.1	28.7	38.7	36.6	43.4	55.5	56.1	43.2	66.0	68.8
	$\mathcal{D}_{\text{GOLD}}$	1	49.1	47.7	47.7	18.7	25.5	25.5	49.6	35.1	35.1	17.4	52.6	52.6
		2	67.8	73.0	61.2	25.2	29.7	24.1	38.1	36.6	37.0	34.6	62.2	59.4
		5	59.3	80.9	69.6	39.3	35.2	31.0	69.2	68.6	66.6	45.4	65.5	63.7
		10	76.0	84.7	75.6	39.6	38.8	37.1	69.3	68.8	68.9	55.8	70.4	68.6
NEO	$\mathcal{D}_{\text{TRAIN}}$	1	49.2	33.8	33.8	12.8	20.2	20.2	25.5	36.5	36.5	11.0	57.2	57.2
		2	76.8	81.5	76.3	17.9	26.9	22.7	30.7	55.5	56.5	17.6	52.6	42.2
		5	65.1	80.8	66.1	19.0	29.2	25.1	40.0	55.9	52.5	25.2	66.4	61.8
		10	69.8	84.1	69.7	12.7	33.7	31.9	69.6	59.3	63.4	41.6	70.6	69.0
	$\mathcal{D}_{\text{GOLD}}$	1	49.3	48.3	48.3	18.5	20.6	20.6	48.3	34.8	34.8	18.2	56.4	56.4
		2	75.1	82.6	78.5	19.7	27.4	22.8	69.3	64.7	64.7	27.8	54.0	44.5
		5	73.2	82.9	71.6	19.2	30.2	26.4	68.7	67.2	65.8	50.4	66.0	61.6
		10	72.4	85.8	71.8	15.4	37.0	34.5	69.8	68.8	68.9	45.2	72.8	68.8

context window size (up to 1024 word pieces), we evaluate the accuracy of the LM in the n -shot setting for all the datasets, where $n = 1, 2, 5, 10$. We use 10 different permutations for each set of randomly selected training samples and use 5 different sets for each experiment, and we report the average accuracy.

As formulated in Section 2, the main insight of GUNDAM is to use $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{TRAIN}}$, instead of $\mathcal{D}_{\text{TRAIN}}$, and thus we use $\mathcal{D}_{\text{GOLD}}$ to denote applying GUNDAM. We summarize the design space, introduced in Section 3.3, as follows. (i) GUNDAM can collaborate with different non-parametric retrievers. Here, we evaluate the performance of GUNDAM in collaboration with three widely applied retrievers. The first one is the random retriever (denoted as RAN) where input demonstrations are randomly selected from the sufficient plug-in database when applying GUNDAM, or the whole training samples when not. The second one is the similarity-based retriever [31, 12] (denoted SIM) which selects relevant demonstrations in terms of the cosine similarity metric. The third one is the diversity-based retriever [38] (denoted as DIV) which selects similar and diverse demonstrations in terms of maximal marginal relevance [3]. (ii) GUNDAM could run the sufficiency tree search algorithm multiple rounds. Here, we perform GUNDAM by running the sufficiency tree search algorithm for 1, 5, 10 rounds.

We provide detailed descriptions of retrievers and implementations in Appendix A4.1. All the codes will be released at publication.

4.2 Performance Comparisons

Table 1 summarizes our main results, where we run the sufficiency tree search algorithm for 5 rounds. We further evaluate the impact of the number of rounds, in terms of both accuracy metric and the size of the sufficient plug-in set in Figure 4. To illustrate the essence of the necessary plug-in data miner, we introduce an ablation study where we duplicate the training dataset, and report the performance changes in Table 2. Our findings are summarized as follows.

GUNDAM is an effective data compressor. By combining Table 1 and Figure 4, we can see that we can save almost half of training samples and, meanwhile, achieve a better or comparable performance in most cases. These results verify the superiority of GUNDAM as an unsupervised core data selector.

Table 2: Results of robustness study where we duplicate the training dataset. We evaluate NEO’s performance on the 2,5,10-shot settings and report the average percentage change compared to using the original training dataset.

$\Psi_{LM}(\cdot)$	$\tilde{\mathcal{D}}_{\text{TRAIN}}$	SST-2			SST-5			COLA			TREC		
		RAN	SIM	DIV	RAN	SIM	DIV	RAN	SIM	DIV	RAN	SIM	DIV
NEO	$\mathcal{D}_{\text{TRAIN}}^D$	-4%	-9%	-1%	-7%	-12%	-2%	-9%	-13%	-2%	-8%	-15%	-2%
	$\mathcal{D}_{\text{GOLD}}^D$	-0%	-0%	-0%	-1%	-1%	-0%	-0%	-1%	-0%	-0%	-1%	-1%

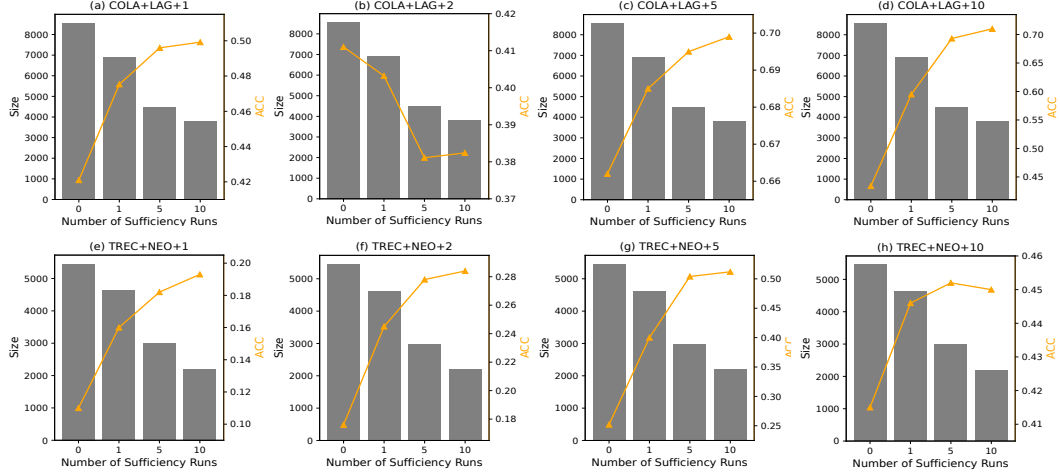


Figure 4: Results comparisons of using different numbers of rounds of sufficient tree-search algorithm in terms of accuracy (denoted as ACC) and the size of the resulting sufficient plug-in data (denoted as Size).

GUNDAM is an effective input demonstration filter. From Table 2, we can observe that both random and similarity-based retrievers are strongly influenced by the duplicated dataset, because the retrieved demonstrations could be overlapped especially when applying a similarity-based retriever (since the similarity scores are assigned individually). In this case, GUNDAM serves as an effective filter to remove the unnecessary part from the input, which can largely benefit these retrievers.

GUNDAM designed on the set level works well when the number of shots is large. From Table 1, good performance of GUNDAM often occur when the number of shots is large, especially when the LM is LAR. One possible explanation is that our method acting on the set level, consider the input demonstrations as a whole, and thus could address potential reciprocal inhibitions among data.

GUNDAM’s performance is conditioned on the LM and the retriever. Table 1 also tells us that the performance of GUNDAM is conditioned on the LM and the retriever. Roughly speaking, GUNDAM can benefit the LMs MED, LAR, NEO, especially NEO, and the retrievers RAN, SIM, DIV. One possible explanation is that GUNDAM is developed based on the capability of the LMs (e.g., reasoning skills and stored knowledge), and the larger LMs usually hold more powerful capabilities.

We provide detailed descriptions of evaluating GUNDAM in the context of supervised learning setting in Appendix A2.4, and a time-complexity evaluation in Appendix A4.5. We also give a case study based on an artificial dataset to further illustrate how GUNDAM works in Appendix A4.4.

5 Conclusion and Future Work

In this paper, we propose a novel LM framework GUNDAM that answers the questions of what kind of data are highly effective and how to find them efficiently. Our experimental results show that GUNDAM could broadly benefit many LMs with the collaborations of either similarity or diversity-based retrievers. In the future, it would be interesting to further investigate the performance of GUNDAM with larger LMs, e.g., GPT-3, and explore more use cases of GUNDAM in data safety and data management.

References

- [1] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Jaime Carbonell and Jade Goldstein. The use of mmr, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*, pages 335–336, 1998.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [6] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, and Zhifang Sui. A survey for in-context learning. *arXiv preprint arXiv:2301.00234*, 2022.
- [7] Liat Ein Dor, Alon Halfon, Ariel Gera, Eyal Shnarch, Lena Dankin, Leshem Choshen, Marina Danilevsky, Ranit Aharonov, Yoav Katz, and Noam Slonim. Active learning for bert: An empirical study. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7949–7962, 2020.
- [8] Dan Feldman. Introduction to core-sets: an updated survey. *arXiv preprint arXiv:2011.09384*, 2020.
- [9] Yarín Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.
- [10] Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. *arXiv preprint arXiv:2012.15723*, 2020.
- [11] Daniel Gissin and Shai Shalev-Shwartz. Discriminative active learning. *arXiv preprint arXiv:1907.06347*, 2019.
- [12] Hila Gonen, Srini Iyer, Terra Blevins, Noah A Smith, and Luke Zettlemoyer. Demystifying prompts in language models via perplexity estimation. *arXiv preprint arXiv:2212.04037*, 2022.
- [13] Chengcheng Guo, Bo Zhao, and Yanbing Bai. Deepcore: A comprehensive library for coreset selection in deep learning. In *Database and Expert Systems Applications: 33rd International Conference, DEXA 2022, Vienna, Austria, August 22–24, 2022, Proceedings, Part I*, pages 181–195. Springer, 2022.
- [14] Vivek Gupta, Akshat Shrivastava, Adithya Sagar, Armen Aghajanyan, and Denis Savenkov. Retronlu: Retrieval augmented task-oriented semantic parsing. *arXiv preprint arXiv:2109.10410*, 2021.
- [15] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Few-shot learning with retrieval augmented language models. *arXiv preprint arXiv:2208.03299*, 2022.
- [16] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *ICLR*, 2019.
- [17] Itay Levy, Ben Bogin, and Jonathan Berant. Diverse demonstrations improve in-context compositional generalization. *arXiv preprint arXiv:2212.06800*, 2022.

- [18] David D Lewis. A sequential algorithm for training text classifiers: Corrigendum and additional data. In *Acm Sigir Forum*, volume 29, pages 13–19. ACM New York, NY, USA, 1995.
- [19] Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? *arXiv preprint arXiv:2101.06804*, 2021.
- [20] Pekka Malo, Ankur Sinha, Pekka Korhonen, Jyrki Wallenius, and Pyry Takala. Good debt or bad debt: Detecting semantic orientations in economic texts. *Journal of the Association for Information Science and Technology*, 65(4):782–796, 2014.
- [21] Bo Pang and Lillian Lee. A sentimental education: Sentiment analysis using subjectivity summarization based on minimum cuts. *arXiv preprint cs/0409058*, 2004.
- [22] Panupong Pasupat, Yuan Zhang, and Kelvin Guu. Controllable semantic parsing via retrieval augmentation. *arXiv preprint arXiv:2110.08458*, 2021.
- [23] Judea Pearl. *Causality: models, reasoning, and inference*, 1980.
- [24] Judea Pearl. *Causality*. Cambridge university press, 2009.
- [25] Linlu Qiu, Peter Shaw, Panupong Pasupat, Tianze Shi, Jonathan Herzig, Emily Pitler, Fei Sha, and Kristina Toutanova. Evaluating the impact of model scale for compositional generalization in semantic parsing. *arXiv preprint arXiv:2205.12253*, 2022.
- [26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
- [28] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- [29] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. Learning to retrieve prompts for in-context learning. *arXiv preprint arXiv:2112.08633*, 2021.
- [30] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013.
- [31] Taylor Sorensen, Joshua Robinson, Christopher Michael Rytting, Alexander Glenn Shaw, Kyle Jeffrey Rogers, Alexia Pauline Delorey, Mahmoud Khalil, Nancy Fulda, and David Wingate. An information-theoretic approach to prompt engineering without ground truth labels. *arXiv preprint arXiv:2203.11364*, 2022.
- [32] Eliza Strickland. Andrew ng: Farewell, big data. *IEEE Spectrum*, Mar, 2022.
- [33] Ellen M Voorhees and Dawn M Tice. Building a question answering test collection. In *Proceedings of the 23rd annual international ACM SIGIR conference on Research and development in information retrieval*, pages 200–207, 2000.
- [34] Shuohang Wang, Yichong Xu, Yuwei Fang, Yang Liu, Siqi Sun, Ruochen Xu, Chenguang Zhu, and Michael Zeng. Training data is more valuable than you think: A simple and effective method by retrieving from training data. *arXiv preprint arXiv:2203.08773*, 2022.
- [35] Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*, 2018.
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

- [37] Michihiro Yasunaga, Armen Aghajanyan, Weijia Shi, Rich James, Jure Leskovec, Percy Liang, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. Retrieval-augmented multimodal language modeling. *arXiv preprint arXiv:2211.12561*, 2022.
- [38] Xi Ye, Srinivasan Iyer, Asli Celikyilmaz, Ves Stoyanov, Greg Durrett, and Ramakanth Pasunuru. Complementary explanations for effective in-context learning. *arXiv preprint arXiv:2211.13892*, 2022.
- [39] Kōsaku Yosida. *Functional analysis*. Springer Science & Business Media, 2012.
- [40] Yury Zemlyanskiy, Michiel de Jong, Joshua Ainslie, Panupong Pasupat, Peter Shaw, Linlu Qiu, Sumit Sanghai, and Fei Sha. Generate-and-retrieve: use your predictions to improve retrieval for semantic parsing. *arXiv preprint arXiv:2209.14899*, 2022.

A1 A Family of Analysis on Data Relationships

We begin by describing some key notations and the assumption used in the paper.

Notation. Let X, Z denote variables for the input question and the (previous) plugged-in data. We use Y , a boolean variable, to denote whether the answer to the input question is right. Concretely, we use $Y_x = 1$ to denote $Y = 1|X = x$, meaning that the LM generates the right answer to the input question x . Similarly, $Y_x = 0$, equivalent to $Y = 0|X = x$, indicates that the LM produces the wrong answer to x . For clarity, we introduce S , a variable to record the original status of the LM before *new* plug-in and unplug operations (denoted as $\text{plug}(\cdot)$ and $\text{unplug}(\cdot)$ respectively), e.g., $Z = ((x, y)), S = (Y_x = 1)$ means that without plugging-in any new data or unplugging any plugged-in data, the plugged-in data is (x, y) and the LM’s performance is $Y_x = 1$.

Assumption A1. We assume that for any data point (e.g., (x, y)), plugging-in itself can guarantee that the LM’s answer to x is always correct. Namely, we have: $Y_x = 1|\text{plug}((x, y)); Z, S$ always holds for any values assigned to Z and S .

A1.1 Data Relationships on Instance level

In this subsection, we consider two instances, denoted as (x_n, y_n) and (x_m, y_m) .

Sufficiency relationship is proposed to evaluate whether plugging-in one data point is *sufficient* to enable the LM to generate the right answer to the other one. Formally, we have:

Definition A1 (Sufficient Plug-in Instance). Given tuple (X, Y, Z, S) , we say that the data point (x_n, y_n) is a sufficient plug-in instance for (x_m, y_m) (namely plugging-in (x_n, y_n) is sufficient to get the right answer to x_m), if the following equation holds:

$$Y_{x_m} = 1|\text{plug}((x_n, y_n)); Z = \emptyset, S = (Y_{x_m} = 0). \quad (11)$$

It indicates that before plugging-in (x_n, y_n) , there is no plugged-in data (i.e., $Z = \emptyset$) and the LM’s answer to x_m is wrong (i.e., $S = (Y_{x_m} = 0)$), and when we plug-in (x_n, y_n) (i.e., $\text{plug}((x_n, y_n))$), then it would correct the LM’s answer to x_m (i.e., $Y_{x_m} = 1$).

Example A1. Let x_m, x_n be *Which country does Sherlock Holmes live?* and *Which city does Sherlock Holmes live?* Suppose that without any plugged-in data, the LM generates wrong answers for the above two questions, e.g., *Sherlock Holmes lives in the United States* for x_m and *Sherlock Holmes lives in New York* for x_n . Then, after telling the LM the right answer of x_n (e.g., y_n is *Sherlock Holmes lives in London*), then the LM can infer the right answer of x_m (e.g., y_m is *Sherlock Holmes lives in the United Kingdom*). In this case, the LM is using the city where Sherlock Holmes lives to infer the country in which he lives.

Necessity relationship is proposed to evaluate whether it is *necessary* to keep one plugged-in data point in terms of maintaining the right answer to the other one. Formally, we have:

Definition A2 (Necessary Plug-in Instance). Given tuple (X, Y, Z, S) , we say that the data point (x_n, y_n) is a necessary plug-in instance for (x_m, y_m) (namely plugging-in (x_n, y_n) is necessary to get the right answer to x_m), if the following equation holds:

$$Y_{x_m} = 0|\text{unplug}((x_n, y_n)); Z = ((x_n, y_n)), S = (Y_{x_m} = 1). \quad (12)$$

It means that before unplugging (x_n, y_n) , there is plugged-in data (x_n, y_n) (i.e., $Z = ((x_n, y_n))$) and the LM’s answer to x_m is right (i.e., $S = (Y_{x_m} = 1)$), and when we unplug (x_n, y_n) (i.e., $\text{unplug}((x_n, y_n))$), then it would cause the LM’s answer to x_m being wrong (i.e., $Y_{x_m} = 0$).

Example A2. Let x_m, x_n be *Which city does Sherlock Holmes live?* and *What is the detailed address of Sherlock Holmes lives?* Suppose that the LM knows nothing about Sherlock Holmes without previous plugged-in (x_n, y_n) (where y_n is *221B Baker Street, London*). After plugging-in (x_n, y_n) , the LM can produce a right answer y_m (where y_m is *Sherlock Holmes live in London*) to x_m . If unplugging (x_n, y_n) , the LM would produce a wrong answer to x_m , e.g., *Sherlock Holmes live in New York*.

From the above description, any data point (x_n, y_n) itself is a sufficient plug-in instance for answering x_n , but it may not be a necessary plug-in instance, as other plugged-in data might always be able to maintain the right answer to x_n . Therefore, ideally, when we aim to ensure the LM’s performance,

a principle solution is to extract plug-in data that are both sufficient and necessary, to guarantee accuracy meanwhile avoiding data redundancy.

Definition A3 (Sufficient and Necessary Plug-in Instance). Given tuple (X, Y, Z, S) , we say that data point (x_n, y_n) is a sufficient and necessary plug-in instance for (x_m, y_m) (namely plugging-in (x_n, y_n) is both sufficient and necessary to get the right answer to x_m), if the following equation holds:

$$\begin{aligned} & \left(Y_{x_m} = 1 | \text{plug}((x_n, y_n)); Z = \emptyset, S = (Y_{x_m} = 0) \right) \\ & \wedge \left(Y_{x_m} = 0 | \text{unplug}((x_n, y_n)); Z = ((x_n, y_n), S = (Y_{x_m} = 1)) \right), \end{aligned} \quad (13)$$

which indicates that plugging-in data point (x_n, y_n) can respond to the LM's answering x_m in both ways.

Although neither of the above two quantities (i.e., sufficiency and necessity) is sufficient for determining the other one, they are not entirely independent, as shown in the following lemma.

Lemma A1. Supposing that we only consider using the data point (x_n, y_n) as the plug-in data, and only care about the LM's performance regarding the input question x_m , then overall there are only two situations here: (i) (x_n, y_n) is plugged-in, and (ii) (x_n, y_n) is not plugged-in. Based on the above assumption, we re-write (i) as plugging-in (x_n, y_n) when there is no plugged-in data (i.e., $\text{plug}((x_n, y_n)); Z = \emptyset, S = (Y_{x_m} = 0)$), and re-write (ii) as unplugging (x_n, y_n) when there is plugged-in data (x_n, y_n) (i.e., $\text{unplug}((x_n, y_n)); Z = ((x_n, y_n)), S = (Y_{x_m} = 1)$). For simplicity, we use Z^* and Z to denote (i) and (ii) respectively; and we use Y^* and Y to denote $Y_{x_1} = 1$ and $Y_{x_1} = 0$. Then, we have: $Z^* \vee Z = \text{true}$, $Z^* \wedge Z = \text{false}$, $Y^* \vee Y = \text{true}$, $Y^* \wedge Y = \text{false}$.

We define PS as the probability of sufficient plug-in data as:

$$\text{PS} := \Pr(Y_{x_m} = 1 | \text{plug}((x_n, y_n)); Z = \emptyset, S = (Y_{x_m} = 0)) = \Pr(Y^* | Z^*). \quad (14)$$

We define PN as the probability of necessary plug-in data as:

$$\text{PN} := \Pr(Y_{x_m} = 0 | \text{unplug}((x_n, y_n)); Z = ((x_n, y_n)), S = (Y_{x_m} = 1)) = \Pr(Y | Z). \quad (15)$$

We further define PNS as the probability of sufficient and necessary plug-in data as:

$$\text{PNS} := \Pr(Y^* | Z^*, Y | Z). \quad (16)$$

Then, PS, PN, PNS satisfy the following relationship:

$$\text{PNS} = \Pr(Y, Z) \cdot \text{PS} + \Pr(Y^*, Z^*) \cdot \text{PN}. \quad (17)$$

Proof. From the above description of Y^*, Y, Z^*, Z , we can write:

$$\begin{aligned} Y^* | Z^* \wedge Y | Z &= (Y^* | Z^* \wedge Y | Z) \wedge (Z \vee Z^*) \\ &= (Y^* | Z^* \wedge Y \wedge Z) \vee (Y | Z \wedge Y^* \wedge Z^*). \end{aligned} \quad (18)$$

Taking probabilities on both sides and using the disjointedness of Z^* and Z , we have:

$$\begin{aligned} \text{PNS} &= \Pr(Y^* | Z^*, Y | Z) = \Pr(Y | Z, Y^*, Z^*) + \Pr(Y^* | Z^*, Y, Z) \\ &= \Pr(Y, Z) \cdot \text{PS} + \Pr(Y^*, Z^*) \cdot \text{PN}. \end{aligned} \quad (19)$$

□

A1.2 Data Relationships on Set level

In this subsection, we first extend Definitions A1 and A2 to the set level as:

Definition A4 (Sufficient Plug-in Set). Given tuple (X, Y, Z) , we say that the input set $\mathcal{D}_{\text{IN}}$ is a sufficient plug-in set for output set $\mathcal{D}_{\text{OUT}}$ (namely plugging-in $\mathcal{D}_{\text{IN}}$ is sufficient to get the right answer to any data point in $\mathcal{D}_{\text{OUT}}$), if the following equation holds:

$$Y_{\{x_n | x_n \in \mathcal{D}_{\text{OUT}}\}} = \mathbf{1}_{N_{\text{OUT}}} | \text{plug}(\mathcal{D}_{\text{IN}}); Z = \emptyset, \quad (20)$$

where $N_{\text{OUT}} = |\mathcal{D}_{\text{OUT}}|$. It indicates that when plugging-in $\mathcal{D}_{\text{IN}}$ (i.e., $\text{plug}(\mathcal{D}_{\text{IN}})$), then it would guarantee that the LM's answer to any input question in $\mathcal{D}_{\text{OUT}}$ is right (i.e., $Y_{\{x_n | x_n \in \mathcal{D}_{\text{OUT}}\}} = \mathbf{1}_{N_{\text{OUT}}}$).

Definition A5 (Necessary Plug-in Set). Given tuple (X, Y, Z, S) , we say that the input set $\mathcal{D}_{\text{IN}}$ is a necessary plug-in set for output set $\mathcal{D}_{\text{OUT}}$ (namely plugging-in **any** subset of $\mathcal{D}_{\text{IN}}$, denoted as $\mathcal{D}'_{\text{IN}}$, is necessary to get the right answer to **at least one** data point in $\mathcal{D}_{\text{OUT}}$), if the following equation holds:

$$Y_{(\{x_n | x_n \in \mathcal{D}_{\text{OUT}}\})} \neq \mathbf{1}_{N_{\text{OUT}}} | \text{unplug}(\mathcal{D}'_{\text{IN}}); Z = \mathcal{D}_{\text{IN}}, S = (Y_{(\{x_n | x_n \in \mathcal{D}_{\text{OUT}}\})} = \mathbf{1}_{N_{\text{OUT}}}), \quad (21)$$

where $N_{\text{OUT}} = |\mathcal{D}_{\text{OUT}}|$. It means that before unplugging any subset of $\mathcal{D}_{\text{IN}}$, there is plugged-in data $\mathcal{D}_{\text{IN}}$ (i.e., $Z = \mathcal{D}_{\text{IN}}$) and the LM's answer to any question in $\mathcal{D}_{\text{OUT}}$ is right (i.e., $S = (Y_{(\{x_n | x_n \in \mathcal{D}_{\text{OUT}}\})} = \mathbf{1}_{N_{\text{OUT}}})$), and when we unplug any subset of $\mathcal{D}_{\text{IN}}$ (i.e., $\text{unplug}(\mathcal{D}'_{\text{IN}})$), then it would cause the LM's answer to at least one input question in $\mathcal{D}_{\text{OUT}}$ being wrong (i.e., $Y_{(\{x_n | x_n \in \mathcal{D}_{\text{OUT}}\})} \neq \mathbf{1}_{N_{\text{OUT}}}$).

From the above descriptions, one can see that when we say one set is a sufficient plug-in set, we require that the overall set of data points is sufficient; when we say one set is a necessary plug-in set, we require that each data point in the set is necessary.

Example A3. Let $\mathcal{D}_{\text{OUT}} = \{(x_m, y_m)\}$ and $\mathcal{D}_{\text{IN}} = \{(x_i, y_i), (x_j, y_j)\}$. We assign x_m and y_m as Which country does Sherlock Holmes live? and Sherlock Holmes lives in the United Kingdom. Let x_i and y_i denote Which street does Sherlock Holmes live? and Baker street. We assign x_j and y_j as Where is Baker street? and Baker street is located in London. Supposing that the LM does not know that Baker street is located in the United Kingdom, then solely plugging-in either (x_i, y_i) or (x_j, y_j) is not sufficient for the LM to get the right answer y_m to the input question x_m . In this regard, it is easy to derive that $\mathcal{D}_{\text{IN}}$ is both sufficient and necessary plug-in set for $\mathcal{D}_{\text{OUT}}$: (i) plugging-in $\mathcal{D}_{\text{IN}}$ is sufficient to maintain the right answer for $\mathcal{D}_{\text{OUT}}$; and (ii) unplugging any subset of $\mathcal{D}_{\text{IN}}$ can not maintain the right answer for $\mathcal{D}_{\text{OUT}}$.

Then, we investigate the following problem: how to extract a subset from the given dataset $\mathcal{D}$ that is both sufficient and necessary to represent $\mathcal{D}$. Formally, we have:

Definition A6 (Golden Plug-in Set). Given tuple (X, Y, Z) and dataset $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$, according to the assumption posted in Appendix A1, we have that $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N | \text{plug}(\mathcal{D})$; $Z = \emptyset$ holds, which means that plugging-in all the data points in $\mathcal{D}$ is able to guarantee the LM getting the right answer to any input question in $\mathcal{D}$. Then, we say that $\mathcal{D}$ can be partitioned into two parts: one is the golden plug-in part, denoted as $\mathcal{D}_{\text{GOLD}}$, and the other part is denoted as $\overline{\mathcal{D}}_{\text{GOLD}}$, if following conditions are satisfied:

- (i) $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N | \text{plug}(\mathcal{D}_{\text{GOLD}})$; $Z = \emptyset$ holds, where $\mathbf{1}_N$ and $\mathbf{0}_N$ denotes N -dimensional vectors whose elements are all 1s and 0s. It indicates that plugging-in $\mathcal{D}_{\text{GOLD}}$ alone is sufficient for maintaining $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N$.
- (ii) $Y_{(x_1, \dots, x_N)} \neq \mathbf{1}_N | \text{unplug}(\mathcal{D}'_{\text{GOLD}} \cup \overline{\mathcal{D}}_{\text{GOLD}})$; $Z = \mathcal{D}$ holds for any subset of $\mathcal{D}_{\text{GOLD}}$ (denoted as $\mathcal{D}'_{\text{GOLD}}$). It indicates that if $\overline{\mathcal{D}}_{\text{GOLD}}$ would be unplugged, then unplugging any data point in $\mathcal{D}_{\text{GOLD}}$ would make the plugged-in data not sufficient to keep the answers accurate for all the input questions. Namely, plugging-in $\mathcal{D}_{\text{GOLD}}$ is necessary for maintaining $Y_{(x_1, \dots, x_N)} = \mathbf{1}_N$.

Example A4. If we merge $\mathcal{D}_{\text{IN}}$ and $\mathcal{D}_{\text{OUT}}$ exemplified in Example A3 into one set $\mathcal{D}$, namely let $\mathcal{D} = \mathcal{D}_{\text{IN}} \cup \mathcal{D}_{\text{OUT}}$, then in this case, it is easy to derive that $\mathcal{D}_{\text{IN}}$ is golden plug-in data for $\mathcal{D}$.

Proposition A1. For the given dataset $\mathcal{D}$, $\mathcal{D}_{\text{GOLD}}$ is a golden plug-in set of $\mathcal{D}$, iff $\mathcal{D}_{\text{GOLD}}$ is a sufficient plug-in set of $\mathcal{D}$ and $\overline{\mathcal{D}}_{\text{GOLD}}$ is a necessary plug-in set of $\mathcal{D}$.

Proof. Firstly, it is simple to show that condition (i) in Definition A6 is equivalent to the sufficiency in Definition A4. Both of them can reach that

$$Y_{(\{x_n | x_n \in \mathcal{D}\})} = \mathbf{1}_N | \text{plug}(\mathcal{D}_{\text{GOLD}}); Z = \emptyset, \quad (22)$$

which indicates that plugging-in $\mathcal{D}_{\text{GOLD}}$ is sufficient to guarantee the LM's answer to any input question in $\mathcal{D}$. Secondly, we re-write condition (ii) in Definition A6 as:

$$Y_{(\{x_n | x_n \in \mathcal{D}\})} \neq \mathbf{1}_N | \text{unplug}(\mathcal{D}'_{\text{GOLD}}); Z = \mathcal{D}_{\text{GOLD}}, S = (Y_{(\{x_n | x_n \in \mathcal{D}\})} = \mathbf{1}_N), \quad (23)$$

where $\mathcal{D}'_{\text{GOLD}}$ can be any subset of $\mathcal{D}_{\text{GOLD}}$. It means that any data point is necessary to maintain the LM's right answers to the input questions in $\mathcal{D}$, namely $\mathcal{D}_{\text{GOLD}}$ is a necessary plug-in set for $\mathcal{D}$.

Similarly, one can directly combine sufficiency and necessity to derive conditions (i) and (ii). $\square$

Algorithm A1 Sufficiency Tree Search Algorithm in GUNDAM

Input: Input dataset $\mathcal{D}_{\text{IN}} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^{N_{\text{IN}}}$.

Output: A sufficient plug-in set of $\mathcal{D}_{\text{IN}}$, denoted as $\mathcal{D}_{\text{OUT}}$.

```
1: Initialize  $k = 1$  and  $\mathcal{W}_0 = \{\mathcal{W}_n = \{(\mathbf{x}_n, \mathbf{y}_n)\} | (\mathbf{x}_n, \mathbf{y}_n) \in \mathcal{D}_{\text{IN}}\}$ .
2: repeat
3:   Initialize  $\mathcal{W}_k = \emptyset$ .
4:   for each pair  $(\mathcal{W}_i, \mathcal{W}_j)$  where  $\mathcal{W}_i, \mathcal{W}_j \in \mathcal{W}_{k-1}$  do
5:     Check  $Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \mathcal{W}_j\})} = \mathbf{1}_{|\mathcal{W}_j|} \text{plug}(\mathcal{W}_i); Z = \emptyset$  (a).
6:     Check  $Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \mathcal{W}_i\})} = \mathbf{1}_{|\mathcal{W}_i|} \text{plug}(\mathcal{W}_j); Z = \emptyset$  (b).
7:     Case I (both (a) and (b) hold), if  $|\mathcal{W}_i| \geq |\mathcal{W}_j|$ , append  $\mathcal{W}_j$  to  $\mathcal{W}_k$ ; otherwise, append  $\mathcal{W}_i$ 
      to  $\mathcal{W}_k$ .
8:     Case II (either one of (a) and (b) holds), if (a) holds, append  $\mathcal{W}_i$  to  $\mathcal{W}_k$ ; otherwise, append
       $\mathcal{W}_j$  to  $\mathcal{W}_k$ .
9:     Case III (neither (a) nor (b) holds), append  $\mathcal{W}_i \cup \mathcal{W}_j$  to  $\mathcal{W}_k$ .
10:    Remove  $\mathcal{W}_i, \mathcal{W}_j$  from  $\mathcal{W}_{k-1}$ , i.e.,  $\mathcal{W}_{k-1} = \mathcal{W}_{k-1} - \{\mathcal{W}_i, \mathcal{W}_j\}$ .
11:  end for
12:  if  $|\mathcal{W}_{k-1}| = 1$  then
13:    Append only element in  $\mathcal{W}_{k-1}$  to  $\mathcal{W}_k$ .
14:  end if
15:  Grow tree from bottom to top via  $k = k + 1$ .
16: until  $|\mathcal{W}_k| = 1$ , and we assume the current iteration is  $K$ .
17: Let  $\mathcal{W}_{\text{SUFFICIENT}}$  denote only one element (i.e. the root node) in  $\mathcal{W}_K$ .
18: Assign  $\mathcal{D}_{\text{OUT}}$  as  $\mathcal{W}_{\text{SUFFICIENT}}$ , i.e.,  $\mathcal{D}_{\text{OUT}} = \mathcal{W}_{\text{SUFFICIENT}}$ .
```

A2 From Theory to Practice

Since there are two different settings in this paper, as introduced in Section 2, we first introduce how to extract the golden plug-in set from the training dataset in the context of the unsupervised learning in Appendix A2.1, A2.2 and A2.3 and then discuss how to extract the golden plug-in set from the reference dataset regarding the supervised learning on the training dataset in Appendix A2.4.

Aligning with notations introduced in Section 3.2, we use $\mathcal{D}_{\text{IN}} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^{N_{\text{IN}}}$ to denote the input dataset of the following sufficiency tree search algorithm and necessity tree search algorithm, and use $\mathcal{D}_{\text{OUT}}$ to denote the output set. Both the sufficiency tree and the necessity tree grow *from bottom to top*. Let K denote the depth of these trees (i.e., the number of iterations), and we use $k = 1, \dots, K$ to denote the k -th iteration to produce the $(k + 1)$ -th layer of the tree.

A2.1 Mining Sufficient Plug-in set with Tree Search

We begin by introducing our assumption on the transitivity of sufficiency.

Assumption A2. We assume that sufficiency is transitive among set. Formally, given arbitrary three set, denoted as $\mathcal{D}_A, \mathcal{D}_B, \mathcal{D}_C$, if $\mathcal{D}_A$ is a sufficient plug-in set of $\mathcal{D}_B$ and $\mathcal{D}_B$ is a sufficient plug-in set of $\mathcal{D}_C$, then we can derive that $\mathcal{D}_A$ is a sufficient plug-in set of $\mathcal{D}_C$.

The intuition behind the above assumption is that LMs especially large LMs are proven to hold a chain-of-thought reasoning ability [36].

We leverage the transitivity of sufficiency to develop a sufficiency tree search algorithm to efficiently mine a sufficient plug-in set from the given dataset $\mathcal{D}_{\text{IN}}$. We summarize the process of building the sufficiency tree and generate the sufficiency set in Algorithm A1, where the key operations lie in lines 7, 8, and 9 to use the sufficient one to replace each pair of nodes. We notice that the output of the algorithm often includes more than just one possibility, since the sufficient plug-in set of the given dataset is usually not only one.

From the algorithm, we can see that we generate the tree from the bottom (including $|\mathcal{D}_{\text{IN}}|$ nodes) to the top (including one root node) by one-to-one comparisons shown in lines 5 and 6. Thus, the depth of the tree is $O(\log_2^{|\mathcal{D}_{\text{IN}}|})$, and the widths of layers in the tree are $|\mathcal{D}_{\text{IN}}|, |\mathcal{D}_{\text{IN}}|/2, \dots, 1$.

Algorithm A2 Necessity Tree Search Algorithm in GUNDAM

Input: Input dataset $\mathcal{D}_{\text{IN}} = \{(x_n, y_n)\}_{n=1}^{N_{\text{IN}}}$.

Output: A necessary plug-in set of $\mathcal{D}_{\text{IN}}$, denoted as $\mathcal{D}_{\text{OUT}}$.

```
1: Initialize  $k = 1$  and  $\mathcal{H}_0 = \emptyset$ .
2: for each instance  $(x_n, y_n)$  in  $\mathcal{D}_{\text{IN}}$  do
3:   Check  $Y_{(\{x_n, y_n\} \in \mathcal{D}_{\text{IN}})} = \mathbf{1}_{N_{\text{IN}}} | \text{unplug}((x_n, y_n)); Z = \mathcal{D}_{\text{IN}}$  (a).
4:   If (a) holds, let  $\mathcal{H}_n = \{(x_n, y_n)\}$  and append  $\mathcal{H}_n$  to  $\mathcal{H}_0$ .
5: end for
6: repeat
7:   Initialize  $\mathcal{H}_k = \emptyset$ .
8:   for each pair  $(\mathcal{H}_i, \mathcal{H}_j)$  where  $\mathcal{H}_i, \mathcal{H}_j \in \mathcal{H}_{k-1}$  do
9:     Check  $Y_{(\{x_n | x_n \in \mathcal{D}_{\text{IN}}\})} = \mathbf{1}_{N_{\text{IN}}} | \text{unplug}(\mathcal{H}_i \cup \mathcal{H}_j); Z = \mathcal{D}_{\text{IN}}$  (b).
10:    If (b) holds, generate a new node  $\mathcal{H}_i \cup \mathcal{H}_j$ , append it to  $\mathcal{H}_k$ , and assign  $\mathcal{H}_i \cup \mathcal{H}_j$  with
        MAINTAIN signals; otherwise, append  $\mathcal{H}_i$  and  $\mathcal{H}_j$  to  $\mathcal{H}_k$ .
11:   end for
12:   Assign  $\mathcal{H}_{\text{MAX}} = \arg \max_{\mathcal{H} \in \mathcal{H}_k} |\mathcal{H}|$  with MAINTAIN signal.
13:   Remove the nodes without MAINTAIN signals in  $\mathcal{H}_k$ .
14:   Grow tree from bottom to top via  $k = k + 1$ .
15: until  $|\mathcal{H}_k| = 1$  where we assume the iteration is  $K$ .
16: Let  $\mathcal{H}_{\text{UNNECESSARY}}$  denote only one element (i.e. the root node) in  $\mathcal{H}_K$ .
17: Assign  $\mathcal{D}_{\text{OUT}}$  as removing  $\mathcal{H}_{\text{UNNECESSARY}}$  from  $\mathcal{D}_{\text{IN}}$ , i.e.,  $\mathcal{D}_{\text{OUT}} = \mathcal{D}_{\text{IN}} - \mathcal{H}_{\text{UNNECESSARY}}$ .
```

A2.2 Mining Necessary Plug-in set with Tree Search

The intuition behind building the necessity tree is that if unplugging $\mathcal{D}_A$ could cause the answers to at least one input question in $\mathcal{D}_C$ from right to wrong, then unplugging $\mathcal{D}_A \cup \mathcal{D}_B$ also can not maintain the answers to all the data points in $\mathcal{D}_C$ right. In other words, necessity is transitive through the inclusion relation of sets, namely if unplugging a subset would degrade the performance, then unplugging the whole set would also degrade the performance.

We summarize the process of building the necessity tree and generate the necessary set in Algorithm A2, where MAINTAIN signals are introduced as a pruning technique, as we only need to take care of those nodes whose size would increase (i.e., assigning MAINTAIN signals to newly generated nodes in line 10), and the nodes with the maximum size (i.e., assigning a MAINTAIN signal to the node with the maximum size in line 12). From the algorithm, we can see that the computation of each iteration is super costly, as it requires running checker for $O(\mathcal{C}_{N_{\text{IN}}}^2)$ times (where $\mathcal{C}$ is a combination operator), which is infeasible in many real-world scenarios.

Therefore, we propose an alternative algorithm as summarized in Algorithm A3, where we introduce the maximum number of rounds R_{MAX} to trade-off the effectiveness and efficiency. From the algorithm, we can see that for each round, we need to build a tree whose depth is $O(\log_2^{N_{\text{IN}}})$ (which is similar to Algorithm A1). Comparing to Algorithm A2, Algorithm A3 introduces R_{MAX} to control the complexity. If we want to obtain a conceptually necessary plug-in set of the given dataset, then we should assign $R_{\text{MAX}} = +\infty$.

In the following, we prove that by combining our sufficiency tree search algorithm (i.e., Algorithm A1) and necessity tree search algorithm (i.e., either Algorithm A2 or Algorithm A3), the resulting set is a golden plug-in set of $\mathcal{D}_{\text{TRAIN}}$. Formally, we have the following theorem.

Theorem A1. *If we successively apply the sufficiency tree search algorithm and the necessity tree search algorithm (either the original one or the alternative one) on $\mathcal{D}_{\text{TRAIN}}$ to obtain $\tilde{\mathcal{D}}_{\text{TRAIN}}$, then $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a golden plug-in set of $\mathcal{D}_{\text{TRAIN}}$.*

Proof. According to Proposition A1, to prove the above theorem, we only need to prove that $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a sufficient plug-in set of $\mathcal{D}_{\text{TRAIN}}$ and a necessary plug-in set of $\mathcal{D}_{\text{TRAIN}}$. For convenience, we use $\tilde{\mathcal{D}}_{\text{TRAIN}}$ to denote the output of the sufficiency tree search algorithm (i.e., Algorithm A1).

We begin by proving sufficiency. Firstly, according to Assumption A2, for each data point in $\mathcal{D}_{\text{TRAIN}}$, the root node in the tree is a sufficient plug-in set for each leaf node in the tree. Formally, we have

Algorithm A3 Alternative Necessity Tree Search Algorithm in GUNDAM

Input: Input dataset $\mathcal{D}_{\text{IN}} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^{N_{\text{IN}}}$, maximum number of rounds R_{MAX} .

Output: A necessary plug-in set of $\mathcal{D}_{\text{IN}}$, denoted as $\mathcal{D}_{\text{OUT}}$.

```

1: Initialize the number of rounds  $r = 0$ , and the set of unnecessary data  $\mathcal{D}_r = \emptyset$ .
2: repeat
3:   Initialize  $k = 1$  and  $\mathcal{H}_0 = \emptyset$ .
4:   Update input dataset by removing the unnecessary part  $\mathcal{D}_{\text{IN}} = \mathcal{D}_{\text{IN}} - \mathcal{D}_r$ .
5:   for each instance  $(\mathbf{x}_n, \mathbf{y}_n)$  in  $\mathcal{D}_{\text{IN}}$  do
6:     Check  $Y_{(\{\mathbf{x}_n | \mathbf{x}_n' \in \mathcal{D}_{\text{IN}}\})} = \mathbf{1}_{N_{\text{IN}}} | \text{unplug}((\mathbf{x}_n, \mathbf{y}_n)); Z = \mathcal{D}_{\text{IN}}$  (a).
7:     If (a) holds, let  $\mathcal{H}_n = \{(\mathbf{x}_n, \mathbf{y}_n)\}$  and append  $\mathcal{H}_n$  to  $\mathcal{H}_0$ .
8:   end for
9:   repeat
10:    Initialize  $\mathcal{H}_k = \emptyset$ .
11:    for each pair  $(\mathcal{H}_i, \mathcal{H}_j)$  where  $\mathcal{H}_i, \mathcal{H}_j \in \mathcal{H}_{k-1}$  do
12:      Check  $Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \mathcal{D}_{\text{IN}}\})} = \mathbf{1}_{N_{\text{IN}}} | \text{unplug}(\mathcal{H}_i \cup \mathcal{H}_j); Z = \mathcal{D}_{\text{IN}}$  (b).
13:      If (b) holds, generate a new node  $\mathcal{H}_i \cup \mathcal{H}_j$ , append it to  $\mathcal{H}_k$ ; otherwise, append  $\mathcal{H}_i$ 
        and  $\mathcal{H}_j$  to  $\mathcal{H}_k$ .
14:    Remove  $\mathcal{H}_i, \mathcal{H}_j$  from  $\mathcal{H}_{k-1}$ , i.e.,  $\mathcal{H}_{k-1} - \{\mathcal{H}_i, \mathcal{H}_j\}$ .
15:    end for
16:    Grow tree from bottom to top via  $k = k + 1$ .
17:  until  $|\mathcal{H}_k| = 1$  where we assume the iteration is  $K$ .
18:  Let  $\mathcal{H}_{\text{UNNECESSARY}}$  denote only one element (i.e. the root node) in  $\mathcal{H}_K$ .
19:  Update the number of rounds, i.e.,  $r = r + 1$ .
20:  Update  $\mathcal{D}_r$  to include the unnecessary part  $\mathcal{H}_{\text{UNNECESSARY}}$ , i.e.,  $\mathcal{D}_r = \mathcal{D}_r \cup \mathcal{H}_{\text{UNNECESSARY}}$ .
21: until  $|\mathcal{H}_{\text{UNNECESSARY}}| \leq 1$  or  $r \geq R_{\text{MAX}}$ .
22: Assign  $\mathcal{D}_{\text{OUT}}$  as removing  $\mathcal{D}_r$  from  $\mathcal{D}_{\text{IN}}$ , i.e.,  $\mathcal{D}_{\text{OUT}} = \mathcal{D}_{\text{IN}} - \mathcal{D}_r$ .
  
```

$Y_{\{\mathbf{x}_n | \mathbf{x}_n \in \mathcal{D}_{\text{TRAIN}}\}} = \mathbf{1}_N | \text{plug}(\hat{\mathcal{D}}_{\text{TRAIN}}), Z = \emptyset$, meaning that the resulting set $\hat{\mathcal{D}}_{\text{TRAIN}}$ is a sufficient plug-in set of $\mathcal{D}_{\text{TRAIN}}$.

Then, either Algorithm A2 or A3 preserves the sufficiency during the necessity checker, i.e., lines 3 and 9 in Algorithm A2, and lines 6 and 12 in Algorithm A3. In other words, we have:

$$Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \hat{\mathcal{D}}_{\text{TRAIN}}\})} = \mathbf{1}_{\hat{N}} | \text{unplug}(\hat{\mathcal{D}}_{\text{TRAIN}} - \tilde{\mathcal{D}}_{\text{TRAIN}}); Z = \tilde{\mathcal{D}}_{\text{TRAIN}}, \quad (24)$$

where $\hat{N} = |\hat{\mathcal{D}}_{\text{TRAIN}}|$. It can be re-written as $Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \hat{\mathcal{D}}_{\text{TRAIN}}\})} = \mathbf{1}_{\hat{N}} | \text{plug}(\tilde{\mathcal{D}}_{\text{TRAIN}}); Z = \emptyset$, showing that plugging-in $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is sufficient for $\hat{\mathcal{D}}_{\text{TRAIN}}$.

Given Assumption A2, by combining $\hat{\mathcal{D}}_{\text{TRAIN}}$ being a sufficient plug-in set of $\mathcal{D}_{\text{TRAIN}}$ and $\tilde{\mathcal{D}}_{\text{TRAIN}}$ being a sufficient plug-in set of $\hat{\mathcal{D}}_{\text{TRAIN}}$, we arrive at $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a sufficient plug-in set of $\mathcal{D}_{\text{TRAIN}}$.

Next, we investigate necessity. Our goal is to prove unplugging any data point in $\tilde{\mathcal{D}}_{\text{TRAIN}}$ would lead to a degrade on the LM's performance. For convenience, we use $(\mathbf{x}_n, \mathbf{y}_n) \in \tilde{\mathcal{D}}_{\text{TRAIN}}$ to denote an arbitrary data point. If we applying Algorithm A2 to execute the necessary tree search algorithm, then $(\mathbf{x}_n, \mathbf{y}_n)$ must be in $\mathcal{H}_0$, or out of $\mathcal{H}_0$.

If $(\mathbf{x}_n, \mathbf{y}_n)$ is not an element in $\mathcal{H}_0$. Then, according to the computing process of $\mathcal{H}_0$ (shown in lines 5 and 6 in Algorithm A2), unplugging $(\mathbf{x}_n, \mathbf{y}_n)$ it would definitively cause the LM's performance on the input dataset $\hat{\mathcal{D}}_{\text{TRAIN}}$ from $Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \hat{\mathcal{D}}_{\text{TRAIN}}\})} = \mathbf{1}_{\hat{N}}$ to $Y_{(\{\mathbf{x}_n | \mathbf{x}_n \in \hat{\mathcal{D}}_{\text{TRAIN}}\})} \neq \mathbf{1}_{\hat{N}}$.

If $(\mathbf{x}_n, \mathbf{y}_n)$ is an element in $\mathcal{H}_0$, then according to lines 7, 8 and 9 in Algorithm A2, $(\mathbf{x}_n, \mathbf{y}_n)$ must be in $\mathcal{H}_{\text{UNNECESSARY}}$; otherwise, $\mathcal{H}_{\text{UNNECESSARY}} \cup \{(\mathbf{x}_n, \mathbf{y}_n)\}$ should be $\mathcal{H}_{\text{MAX}}$ and always stay in $\mathcal{H}$. until becoming the root node (i.e., $\mathcal{H}_{\text{UNNECESSARY}}$ should be updated to be $\mathcal{H}_{\text{UNNECESSARY}} \cup \{(\mathbf{x}_n, \mathbf{y}_n)\}$). Thus, $(\mathbf{x}_n, \mathbf{y}_n)$ must be in $\mathcal{H}_{\text{UNNECESSARY}}$. However, all the data points in $\mathcal{H}_{\text{UNNECESSARY}}$ are removed from $\tilde{\mathcal{D}}_{\text{TRAIN}}$, causing a contradiction. Hence, unplugging $(\mathbf{x}_n, \mathbf{y}_n)$ would change the LM's performance, namely necessity holds.

Then, we consider the case where we apply Algorithm A3 to execute the necessary tree search.

Algorithm A4 Scaling Up GUNDAM in Unsupervised Learning Setting

Input: Training dataset $\mathcal{D}_{\text{TRAIN}}$, test dataset $\mathcal{D}_{\text{TEST}}$, a non-parametric retriever $\Psi_{\text{RETRIEVE}}(\cdot)$, a LM $\Psi_{\text{LM}}(\cdot)$, number of shots N_{shot} , number of rounds $R_{\text{SUFFICIENT}}$, sufficiency tree searcher $\Psi_{\text{SUFFICIENT}}(\cdot)$, necessary tree searcher $\Psi_{\text{NECESSARY}}(\cdot)$.
Output: Output prediction for test dataset $\hat{y}$.

```
1: Initialize  $r = 1$ , and  $\mathcal{D}_1 = \mathcal{D}_{\text{TRAIN}}$ .
2: repeat
3:   Filter training dataset  $\mathcal{D}_{r+1} = \Psi_{\text{SUFFICIENT}}(\mathcal{D}_r)$  by using Algorithm A1.
4:    $r = r + 1$ .
5: until  $r = R_{\text{SUFFICIENT}}$ .
6: for each input  $x_m \in \mathcal{D}_{\text{TEST}}$  do
7:   Retrieve  $N_{\text{shot}}$  training samples  $\mathcal{D}_{\text{RETRIEVE}} = \Psi_{\text{RETRIEVE}}(x_m, \mathcal{D}_{R_{\text{SUFFICIENT}}})$ .
8:   Filter retrieved samples  $\mathcal{D}_{\text{NECESSARY}} = \Psi_{\text{NECESSARY}}(\mathcal{D}_{\text{RETRIEVE}})$  by using Algorithm A3 (where  $R_{\text{MAX}} = 1$ ).
9:   if  $|\mathcal{D}_{\text{NECESSARY}}| < N_{\text{shot}}$  then
10:    Retrieve new  $N_{\text{shot}} - |\mathcal{D}_{\text{NECESSARY}}|$  samples  $\mathcal{D}'_{\text{RETRIEVE}} = \Psi_{\text{RETRIEVE}}(x_m, \mathcal{D}_{R_{\text{SUFFICIENT}}})$ .
11:     $\mathcal{D}_{\text{NECESSARY}} = \mathcal{D}_{\text{NECESSARY}} \cup \mathcal{D}'_{\text{NECESSARY}}$ .
12:   end if
13:   Make predictions  $\hat{y}_m = \Psi_{\text{LM}}(x_m, \mathcal{D}_{\text{NECESSARY}})$ .
14: end for
```

Similarly, if (x_n, y_n) is not selected by the necessity checker shown in line 6 in Algorithm A3, then unplugging (x_n, y_n) would definitively cause a degrade on the LM's performance.

If (x_n, y_n) is selected by the necessity checker, then (x_n, y_n) must be included in $\mathcal{D}_r$; otherwise, $\mathcal{D}_r$ would continue to update, since the condition of stopping iteration is that there is no or only one unnecessary node. However, all the data points are removed from $\hat{\mathcal{D}}_{\text{TRAIN}}$, causing a contradiction. Hence, unplugging (x_n, y_n) would change the LM's performance, namely necessity holds.

Combining the above sufficiency and necessity, we can conclude that $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a golden plug-in data for $\mathcal{D}_{\text{TRAIN}}$. $\square$

Lemma A2. *If we directly apply the necessary tree search algorithm (either the original one or the alternative one) on $\mathcal{D}_{\text{TRAIN}}$ to obtain $\tilde{\mathcal{D}}_{\text{TRAIN}}$, then $\tilde{\mathcal{D}}_{\text{TRAIN}}$ is a golden plug-in set of $\mathcal{D}_{\text{TRAIN}}$.*

Proof. According to Assumption A1, $\mathcal{D}_{\text{TRAIN}}$ is also a sufficient plug-in set of $\mathcal{D}_{\text{TRAIN}}$, as it satisfies that $Y_{\{x_n | x_n \in \mathcal{D}_{\text{TRAIN}}\}} = \mathbf{1}_N | \text{plug}(\mathcal{D}_{\text{TRAIN}}); Z = \emptyset$.

Then, we can directly follow the proof of Theorem A1, by assigning $\hat{\mathcal{D}}_{\text{TRAIN}} = \mathcal{D}_{\text{TRAIN}}$. $\square$

A2.3 Scaling Up GUNDAM into Real-World Applications

As discussed in Section 3.3, directly applying the necessary tree search algorithm (i.e., either Algorithm A2 or Algorithm A3) is not feasible in practice, because (i) due to limited context window size of the LM, it is usually impractical to plugging-in all the data points in the input dataset (i.e., $\text{plug}(\mathcal{D}_{\text{IN}})$), and (ii) when the input dataset is large, running the algorithm is super time-consuming.

We note that Lemma A2 reveals that our sufficient tree search algorithm (i.e., Algorithm A1) could be regarded as a data filter, because the output dataset of Algorithm A1 is sufficient to represent the input dataset. Therefore, given the training dataset, we can recursively apply the sufficient tree search algorithm to compress the dataset before running the necessary tree search algorithm. Formally, we use $R_{\text{SUFFICIENT}}$ to denote the number of repeating rounds.

To make the necessary tree search algorithm applicable, we incorporate a non-parametric retriever, denoted as $\Psi_{\text{RETRIEVE}}(\cdot)$, that extracts several samples from the filtered training datasets. $\Psi_{\text{RETRIEVE}}(\cdot)$ can be a random retriever that randomly selects samples, or a similarity-based retriever that selects samples similar to the test sample, or a diversity-based retriever that selects samples similar to the test samples meanwhile diverse to retrieved samples. See the formal formulation in Appendix A4.1.

Algorithm A5 Scaling Up GUNDAM in Supervised Learning Setting

Input: Reference dataset $\mathcal{D}_{\text{REFER}}$, training dataset $\mathcal{D}_{\text{TRAIN}}$, a non-parametric retriever $\Psi_{\text{RETRIEVE}}(\cdot)$, a LM $\Psi_{\text{LM}}(\cdot)$, number of shots N_{shot} , number of rounds $R_{\text{SUFFICIENT}}$, sufficiency tree searcher $\Psi_{\text{SUFFICIENT}}(\cdot)$, necessary tree searcher $\Psi_{\text{NECESSARY}}(\cdot)$.
Output: A fine-tuned LM model $\Psi_{\text{LM}}(\cdot)$.

- 1: Initialize $r = 1$, and $\mathcal{D}_1 = \mathcal{D}_{\text{REFER}}$. ▷ Prompt-based (few-shot) fine-tuning
 - 2: **repeat**
 - 3: Filter reference dataset $\mathcal{D}_{r+1} = \Psi_{\text{SUFFICIENT}}(\mathcal{D}_r)$ by using Algorithm A1.
 - 4: $r = r + 1$.
 - 5: **until** $r = R_{\text{SUFFICIENT}}$.
 - 6: **for** each input $x_m \in \mathcal{D}_{\text{TRAIN}}$ **do**
 - 7: Retrieve N_{shot} reference samples $\mathcal{D}_{\text{RETRIEVE}} = \Psi_{\text{RETRIEVE}}(x_m, \mathcal{D}_{R_{\text{SUFFICIENT}}})$.
 - 8: Filter retrieved samples $\mathcal{D}_{\text{NECESSARY}} = \Psi_{\text{NECESSARY}}(\mathcal{D}_{\text{RETRIEVE}})$ by using Algorithm A3 (where $R_{\text{MAX}} = 1$).
 - 9: **if** $|\mathcal{D}_{\text{NECESSARY}}| < N_{\text{shot}}$ **then**
 - 10: Retrieve new $N_{\text{shot}} - |\mathcal{D}_{\text{NECESSARY}}|$ samples $\mathcal{D}'_{\text{RETRIEVE}} = \Psi_{\text{RETRIEVE}}(x_m, \mathcal{D}_{R_{\text{SUFFICIENT}}})$.
 - 11: $\mathcal{D}_{\text{NECESSARY}} = \mathcal{D}_{\text{NECESSARY}} \cup \mathcal{D}'_{\text{NECESSARY}}$.
 - 12: **end if**
 - 13: Make predictions $\hat{y}_m = \Psi_{\text{LM}}(x_m, \mathcal{D}_{\text{NECESSARY}})$.
 - 14: **end for**
 - 15: Update $\Psi_{\text{LM}}(\cdot)$ by minimizing the distances between $\hat{y}_m$ and y_m for each $x_m \in \mathcal{D}_{\text{TRAIN}}$ using Eq. (4).
 - 16: Initialize $r = 1$ and $\mathcal{D}_1 = \mathcal{D}_{\text{TRAIN}}$ ▷ (Zero-shot) fine-tuning
 - 17: **repeat**
 - 18: Filter training dataset $\mathcal{D}_{r+1} = \Psi_{\text{SUFFICIENT}}(\mathcal{D}_r)$ by using Algorithm A1.
 - 19: $r = r + 1$.
 - 20: **until** $r = R_{\text{SUFFICIENT}}$.
 - 21: **for** each input $x_n \in \mathcal{D}_{\text{TRAIN}}$ **do**
 - 22: Make predictions $\hat{y}_n = \Psi_{\text{LM}}(x_n)$.
 - 23: **end for**
 - 24: Update $\Psi_{\text{LM}}(\cdot)$ by minimizing the distances between $\hat{y}_n$ and y_n for each $x_n \in \mathcal{D}_{\text{TRAIN}}$ using Eq. (4).
-

As illustrated in Figure 3(a), we place the retriever between the sufficiency checker and the necessity checker, then we only need to run the necessary tree search algorithm on the retrieved samples, which is acceptable in practice. Also, we use Algorithm A3 as the necessary tree search algorithm and set $R_{\text{MAX}} = 1$ to save the computation cost.

We summarize the above process in Algorithm A4. We first run the sufficient tree search algorithm (i.e., Algorithm A1) for $R_{\text{SUFFICIENT}}$ times, as shown in lines 3 and 4, resulting in a slimming-down dataset $\mathcal{D}_{R_{\text{SUFFICIENT}}}$. And, then for each test sample, we apply the retriever in line 7 to extract N_{shot} samples from $\mathcal{D}_{R_{\text{SUFFICIENT}}}$ before running the necessary tree search algorithm (i.e., Algorithm A3) in line 8. We further guarantee that there are N_{shot} samples in lines 10 and 11.

A2.4 From Unsupervised Learning Setting to Supervised Learning Setting

As introduced in Section 2, the supervised learning setting (i.e., fine-tuning) can be summarized into a bi-level optimization problem. Formally, we are given a reference dataset $\mathcal{D}_{\text{REFER}} = \{(x_q, y_q)\}_{q=1}^Q$, a training dataset $\mathcal{D}_{\text{TRAIN}} = \{(x_n, y_n)\}_{n=1}^N$, and a test dataset $\mathcal{D}_{\text{TEST}} = \{(x_m, y_m)\}_{m=1}^M$. In the outer loop, our goal is to find a high-quality subset of $\mathcal{D}_{\text{REFER}}$ (with a frozen LM $\Psi_{\text{LM}}(\cdot)$), where we adopt the sufficient tree search algorithm (i.e., Algorithm A1) to get it. We denote the resulting set of reference data as $\tilde{\mathcal{D}}_{\text{REFER}}$. In the inner loop, we use a fixed $\tilde{\mathcal{D}}_{\text{REFER}}$ as the set of retrieval to train the LM on $\mathcal{D}_{\text{TRAIN}}$, where we also include a retriever before the necessary tree search algorithm (i.e., Algorithm A3).

We summarize the above process in the black part of Algorithm A5, which is aligned with the process of our unsupervised learning. Here, lines 3, 4, 7, 8, 10, and 11 correspond to the outer loop, and lines 11 and 13 correspond to the inner loop.

This process is also related to prompt-based fine-tuning techniques [10], where the reference data is used as the prompts in fine-tuning.

We also consider the case where there is no reference data available, and in this case, we can only directly fine-tune the LM on the training dataset. Here, GUNDAM is applied to extract a high-quality set of training examples used to fine-tune the LM. This process is aligned with the classical pre-training and fine-tuning paradigm.

We summarize the above process in the gray part of Algorithm A5, where we use the sufficient tree search algorithm (i.e., Algorithm A1) to extract a high-quality subset of $\mathcal{D}_{\text{TRAIN}}$, and directly use it to fine-tune $\Psi_{\text{LM}}(\cdot)$. Here, lines 18 and 19 correspond to the outer loop, and lines 22 and 24 correspond to the inner loop.

A3 Connections to Existing Approaches

A3.1 Connections to Causality

Concepts of sufficiency and necessity have a broad application scope, especially in causality [23, 24], where sufficiency and necessity are proposed to define the causal relationship between two binary variables. Let X and Y denote a pair of variables. Then, the probability of sufficiency measures the capacity of setting $X = \text{true}$ to produce $Y = \text{true}$, while the probability of necessity measures the changing the value of X from $X = \text{true}$ to $X = \text{false}$ would cause the value of Y changing from $Y = \text{true}$ to $Y = \text{false}$.

In this paper, we adopt the concepts of sufficiency and necessity in the context of plugging-in data, where we investigate whether plugging-in certain data points is sufficient or necessary for the given LM to generate the right answers for the input questions. For this purpose, we introduce the plugging-in operation, denoted as $\text{plug}(\cdot)$, to examine the sufficiency and the unplugging operation, denoted as $\text{unplug}(\cdot)$, to examine the necessity. Both of the above operations are similar to the do operation in the causality, denoted as $\text{do}(\cdot)$, which indicates that the system operates under the condition that certain variables are controlled by external forces.

To be more specific, in our setting, the external force can be explained as: we can choose to either plug-in or unplug some data points to change what is already plugged-in for the LM. Our intuition is similar to the counterfactual idea in the casualty, which investigates what if some variables are set with some different values. In our case, we study what if the plugged-in data include some data points that are different from the historical (a.k.a., factual) setting. One significant difference between ours and the counterfactual setting in casualty is that we do not need to estimate the “counterfactual” situations (namely setting plugged-in data with different values), as we can directly do evaluations.

A3.2 Connections to Demonstration Selection

As large LMs have shown great potential in few-shot inference, a core challenge is how to select “shots”. One principal solution is to select some training examples as the extra input, which are named as demonstrations or prompts [17, 19, 6]. One implicit assumption here is that the training set is a support set [39] of test samples. Previous investigations [34, 29] show that plugging-in similar training examples can improve the performance of LMs on test instances. However, as pointed out in [17], these methods only measure each data point in isolation, instead of considering the data points as a whole. In other words, a retriever based on the similarity metric, only operates on the instance level and could select two same data points together as the extra input, which are definitively redundant. To address this issue, a recent paper [17] proposes to consider the diversity among the data points, to avoid the case where too “similar” data points are selected together.

In this paper, our main insight is that the quality of demonstrations would be aware of the LM in use. In other words, a high-quality demonstration for one LM could be a low-quality demonstration for another LM. In light of this, we propose sufficiency and necessity as the new set-level metrics. Our method conveys the following advantages. Firstly, compared to the similarity and diversity metrics, sufficiency and necessity measure the quality of data points depending on the LM in use. Secondly,

Table A1: Result comparisons on text classification datasets for different LM bases (MED, LAR, NEO) with different retrievers (RAN, SIM, DIV) in different shot settings (1, 2, 5, 10). All the results are calculated based on 5 different random seeds, and the bold numbers mean using $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{TRAIN}}$ can obtain at least 2% improvements than using $\mathcal{D}_{\text{TRAIN}}$, or vice versa. See Table 1 for the results on SST-2, SST-5, COLA, and TREC datasets.

$\Psi_{\text{LM}}(\cdot)$	$\tilde{\mathcal{D}}_{\text{TRAIN}}$	n	SUBJ			FPB		
			RAN	SIM	DIV	RAN	SIM	DIV
MED	$\mathcal{D}_{\text{TRAIN}}$	1	51.6	41.1	41.1	27.2	25.3	25.3
		2	47.1	62.8	71.9	27.4	45.8	40.4
		5	51.6	85.8	70.1	26.3	55.9	44.7
		10	62.1	88.0	78.2	27.8	63.1	50.7
	$\mathcal{D}_{\text{GOLD}}$	1	51.7	44.9	44.9	28.2	28.8	28.8
		2	55.9	63.4	74.7	35.5	47.4	37.9
		5	57.5	86.4	69.8	28.3	54.6	47.9
		10	63.5	88.1	77.7	39.6	63.2	49.8
LAR	$\mathcal{D}_{\text{TRAIN}}$	1	51.5	43.6	43.6	33.8	29.9	29.9
		2	58.1	88.3	87.0	27.0	55.4	49.9
		5	66.7	86.2	66.7	27.2	64.3	45.1
		10	48.6	85.9	73.9	47.0	65.5	52.9
	$\mathcal{D}_{\text{GOLD}}$	1	52.5	46.4	46.4	33.8	32.6	32.6
		2	63.1	88.5	86.8	37.5	54.8	47.6
		5	73.4	86.2	66.6	38.9	64.5	48.0
		10	52.0	85.4	73.5	63.5	64.7	50.1
NEO	$\mathcal{D}_{\text{TRAIN}}$	1	52.8	42.1	42.1	54.9	61.6	61.6
		2	48.5	68.3	72.6	53.6	66.8	60.0
		5	51.6	80.5	61.7	28.2	68.2	60.4
		10	48.5	89.1	81.9	49.0	75.8	71.1
	$\mathcal{D}_{\text{GOLD}}$	1	53.2	46.3	46.3	58.1	61.8	61.8
		2	62.6	68.4	73.3	61.4	64.1	58.8
		5	69.4	79.2	61.9	43.2	68.8	62.7
		10	78.7	88.1	81.2	61.4	74.8	71.9

compared to instance-level metrics, our proposed sufficiency and necessity can be extended to the set level, and thus can consider the data points as a whole. In our framework, the “similarity” can be translated as “sufficiency”, meaning that plugging-in the data points can improve the performance of the LM, while the “diversity” can be translated as “necessity”, indicating that each data point should play an irreplaceable role.

A3.3 Connections to Active Learning and Core-set Selection

Core-set selection [8, 13], aiming to select a subset of the most informative training samples, is a long-standing learning problem that can benefit many downstream tasks such as active learning. One prior work [7] has summarized and evaluated existing state-of-the-art active learning approaches for BERT [5], including random sampling, uncertainty-sampling (using entropy metric) [18, 9] and diversity sampling (using diversity metric) [11].

Different from active learning, GUNDAM selects a core set (called golden plug-in data) as the extra inputs instead of using them to train the LM. In this regard, our approach is an unsupervised learning version of core-set selection, which extracts sufficient and necessary training samples on a specific downstream task. And, the “uncertainty” could select the data points with max entropy, independent of the LM. In contrast, “informative training samples” in our framework are defined as those training samples that can benefit the LM’s performance on a specific task.

A4 Experiment

A4.1 Implementation Details

For each dataset, we directly follow the official splits to obtain $\mathcal{D}_{\text{TRAIN}}$ and $\mathcal{D}_{\text{TEST}}$. As for the supervised learning setting, we assign the reference dataset as the training dataset, i.e., $\mathcal{D}_{\text{REFER}} = \mathcal{D}_{\text{TRAIN}}$. In other words, when we apply the prompt-based (few-shot) fine-tuning, as shown in the black part in

Table A2: Result comparisons on directly fine-tuning the LM on golden plug-in data with different rounds (as shown in the gray part of Algorithm A5), on COLA dataset and NEO LM base in the zero-shot setting. Here, $R_{\text{SUFFICIENT}} = 0$ means using the whole training dataset $\mathcal{D}_{\text{TRAIN}}$, and $R_{\text{SUFFICIENT}} = 1$ to 10 means using different $\mathcal{D}_{\text{GOLD}}$. All the results are calculated based on 5 different random seeds, and the bold numbers mean using $\mathcal{D}_{\text{GOLD}}$ as $\mathcal{D}_{\text{TRAIN}}$ can obtain at least 2% improvements than using $\mathcal{D}_{\text{TRAIN}}$. We also separately report the results after fine-tuning the LM on 1000 batches and 2000 batches.

Number of Batches	Number of Rounds $R_{\text{SUFFICIENT}}$										
	0	1	2	3	4	5	6	7	8	9	10
1000	69.2	77.8	69.5	73.1	69.6	70.2	69.8	68.9	69.3	69.1	68.7
2000	67.7	79.3	74.4	70.4	71.5	70.2	70.0	69.0	69.3	69.1	68.7

Algorithm A5, then we extract a high-quality set of training samples as prompts, and fine-tune the LM on the whole training dataset. We also evaluate the performance of directly fine-tuning the LM without the reference dataset, as shown in the gray part of Algorithm A5.

We further introduce detailed descriptions of three different retrievers in our paper. The first one is a random retriever, denoted as RAN, which randomly retrieves samples from the retrieval pool. The second one is a similarity based retriever, denoted as SIM, which retrieves samples similar to the test samples. Formally, let $\mathcal{D}_{\text{RETRIEVE}}$ denote the retrieval pool. Then, for each test sample x_m , the metric of SIM can be written as:

$$\text{SIM}(x_m, x_n) = \text{COS}(\text{TRANSFORMER}(x_m), \text{TRANSFORMER}(x_n)), \text{ where } x_n \in \mathcal{D}_{\text{RETRIEVE}}, \quad (25)$$

where $\text{COS}(\cdot)$ denotes a cosine similarity metric, and $\text{TRANSFORMER}(\cdot)$ denotes a sentence transformer [28]. Here, we directly use the Sentence Transformers library¹ from Hugging Face in our implementation. Then, we are able to retrieve N_{shot} samples with maximum SIM values from $\mathcal{D}_{\text{RETRIEVE}}$. The third one is a diversity based retriever, denoted as DIV, where we adopt the maximal marginal relevance method [3] as the metric of DIV. Formally, we have:

$$\text{DIV}(x_m, x_n) = \text{SIM}(x_m, x_n) - \eta \cdot \max_{x_{n'} \in \mathcal{D}'_{\text{RETRIEVE}}} \text{SIM}(x_m, x_{n'}), \text{ where } x_n \in \mathcal{D}_{\text{RETRIEVE}} - \mathcal{D}'_{\text{RETRIEVE}}, \quad (26)$$

where $\mathcal{D}'_{\text{RETRIEVE}}$ denotes the set of previously selected instances. We can see that DIV prefers the instance that is both similar to the test samples meanwhile distant to previously selected instances. η is a hyper-parameter to balance the above two parts. We set $\eta = 1$ in our experiment.

We list some key hyper-parameters used in our supervised learning setting: the batch size is set as 32, the warm steps is set as 100, the learning rate is set as 5×10^{-5} , and the weight decay is set as 0.01.

A4.2 Additional Experimental Results

We report the results of SST-2, SST-5, COLA, and TREC datasets in Table 1. Here, we further report the results of SUBJ and FPB datasets in Table A1. From the table, we can see that our approach using the selected golden plug-in set, can achieve better or comparable results than using the whole training dataset. Results also indicate that our method can significantly improve the performance of the LM when incorporated with a random retriever. This verifies that our method can truly select high-quality data from the training dataset.

A4.3 Evaluations of the Supervised Learning Setting

As introduced in Appendix A2.4, there are two different supervised learning settings in our paper, corresponding to the prompt-based (few-shot) fine-tuning (shown in the black part in Algorithm A5) and directly fine-tuning (shown in the gray part in Algorithm A5), respectively.

We first evaluate the performance of directly fine-tuning, and report the results in Table A2. From the table, we can see that compared to using the whole training dataset (i.e., $R_{\text{SUFFICIENT}} = 0$) to fine-tune, our method can provide the LM the high-quality training samples, and thus could gain better performance, especially when $R_{\text{SUFFICIENT}}$ is smaller than 4. One possible explanation is that

¹<https://huggingface.co/sentence-transformers>

Table A3: Result comparisons on prompt-based fine-tuning the LM by using the golden plug-in data as the reference data (as shown in the black part in Algorithm A5) with different retrievers (RAN, SIM, DIV) in different shot settings (1, 2, 5, 10). For comparison, we also report the results of using the whole training dataset as the reference data. All the results are calculated based on 5 different random seeds, and the bold numbers mean using $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{REFER}}$ can obtain at least 2% improvements than using $\mathcal{D}_{\text{TRAIN}}$.

$\Psi_{\text{LM}}(\cdot)$	$\tilde{\mathcal{D}}_{\text{REFER}}$	n	COLA		
			RAN	SIM	DIV
NEO	$\mathcal{D}_{\text{TRAIN}}$	1	49.3	68.5	68.5
		2	70.8	63.9	64.3
		5	30.7	54.4	54.0
		10	30.7	50.7	47.6
	$\mathcal{D}_{\text{GOLD}}$	1	76.8	73.4	72.7
		2	74.1	71.3	71.5
		5	71.7	69.4	70.0
		10	71.3	68.5	68.5

when $R_{\text{SUFFICIENT}}$ is larger than 4, then there would be many relatively valuable training samples removed from the training dataset, which would hurt the performance; when $R_{\text{SUFFICIENT}}$ is smaller than 4, then it would be hard for the LM to capture the key information during the training process.

As Table A2 verifies that our method can select high-quality data in terms of (zero-shot) fine-tuning performance. We further evaluate the performance of our method in prompt-based fine-tuning setting. We report the results of running the sufficient tree search algorithm for 5 rounds and fine-tuning on 2000 batches in Table A3. From the table, we can see that using $\mathcal{D}_{\text{GOLD}}$ as $\tilde{\mathcal{D}}_{\text{REFER}}$ can consistently outperform using $\mathcal{D}_{\text{TRAIN}}$ as $\tilde{\mathcal{D}}_{\text{REFER}}$ with different retrievers in different shot settings. One possible explanation is that generally training on the whole samples (i.e., $\mathcal{D}_{\text{TRAIN}}$) and prompting with typical (i.e., high-quality) samples (i.e., $\mathcal{D}_{\text{GOLD}}$) would effectively take advantage of the capability of the LM.

A4.4 Case Study with Artificial Data points generated by Large LMs

Here, we establish a case study to verify the key idea of this paper, i.e., whether a demonstration is high-quality or not should consider the LM in use.

We consider the factual error made by Google Bard in the first demo². Bard said *The James Webb Space Telescope took the very first pictures of a planet outside of our own solar system*. However, the fact is *The Very Large Telescope took the very first pictures of a planet outside of our own solar system*. Based on the above statements, we produce the question *What took the very first pictures of a planet outside of our own solar system?*

Then, we ask the question to gpt-3.5-turbo. The corresponding answer is:

The first pictures of a planet outside of our own solar system were taken by the Hubble Space Telescope in 2008. The planet, named Fomalhaut b, orbits the star Fomalhaut, which is located about 25 light-years away from Earth in the constellation Piscis Austrinus.

We can see that the above answer is wrong. We then prompt the fact into the LM, then the answer is:

The Very Large Telescope took the very first pictures of a planet outside of our own solar system.

The above answer is right, which verifies our Assumption A1. We ask gpt-3.5-turbo to generate some sufficient statements for the fact. The following is its output:

1. The Very Large Telescope made history by capturing the first-ever images of a planet beyond our solar system.
2. Thanks to the advanced technology of the Very Large Telescope, scientists were able to capture the first-ever photographs of an exoplanet.

²<https://www.theverge.com/2023/2/8/23590864/google-ai-chatbot-bard-mistake-error-exoplanet-demo>

3. The groundbreaking images captured by the Very Large Telescope marked a major milestone in the study of exoplanets.
4. The discovery of a planet outside of our solar system was made possible by the cutting-edge capabilities of the Very Large Telescope.
5. The Very Large Telescope’s ability to capture images of distant celestial bodies allowed scientists to observe an exoplanet for the first time in history.

We also evaluate separately using the above statements as a prompt to gpt-3.5-turbo, and either one of them can lead to a right answer. We provide detailed answers as follows.

1. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
2. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
3. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
4. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.
5. The Very Large Telescope took the very first pictures of a planet outside of our own solar system.

We can see, in this case, either one of the statements generated by gpt-3.5-turbo, is a sufficient plug-in instance and a necessary plug-in instance to answer *What took the very first pictures of a planet outside of our own solar system?*

We further evaluate the performance of gpt-j-6b. Without any prompt (i.e., in the zero-shot setting), its answer is:

The Hubble Space Telescope.

We then prompt the above five statements into gpt-j-6b, then the answer is:

1. The first-ever images of a planet beyond our solar system have been captured by the Very Large Telescope in Chile.
2. The Hubble Space Telescope.
3. A team of astronomers led by the University of Arizona’s Michael Liu.
4. The Hubble Space Telescope.
5. The Very Large Telescope.

We can see that only the first or the fifth statement is a sufficient plug-in instance. Combining the results of gpt-j-6b and the results of gpt-3.5-turbo can verify the core insight of our paper: *the measurement over a plug-in instance should consider what LM is in use.*

Furthermore, we also evaluate the performance of three GPT variants used in the paper. We begin by evaluating the zero-shot performance of gpt-neo-1.3B (denoted as NEO in the experiment), and its result is:

The first pictures of a planet outside of our own solar system were taken by the Voyager 1 spacecraft in 1977

We then prompt the above five statements into gpt-neo-1.3B, then the answer is:

1. The very large telescope was built in the early 1990s by the European Southern Observatory (ESO) in Chile.
The Very Large Telescope.
2. The Very Large Telescope in Chile.
3. The Very Large Telescope (VLT) in Chile.
4. The Very Large Telescope.
5. The Very Large Telescope in Chile

The above results show that either one of the latter four statements is a sufficient plug-in instance. The results of gpt2-large (denoted as LAR in the experiment) show that neither of the five statements is a sufficient plug-in instance:

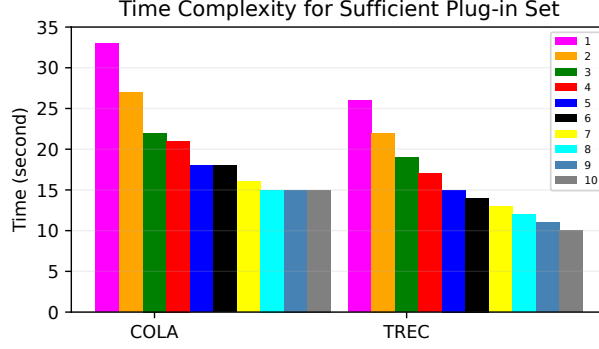


Figure A1: Time comparisons of running the sufficient tree search algorithm on COLA and TREC datasets with different rounds.

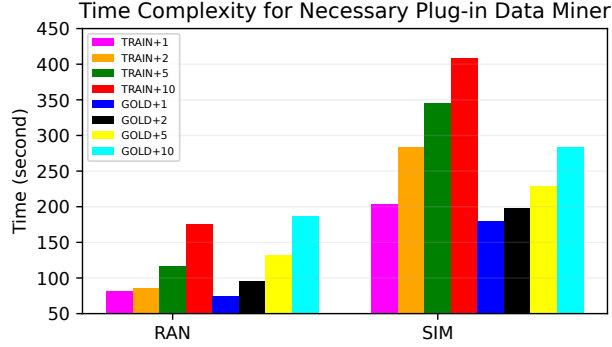


Figure A2: Time comparisons of running the LM NEO in the unsupervised learning setting (including necessary tree search algorithm) on COLA dataset with different retrievers (RAN, SIM) in different shot setting (1, 2, 5, 10). Here, we use TRAIN + X to denote the result of using $\mathcal{D}_{\text{TRAIN}}$ as the retrieval pool in the setting of X -shot inference, and use GOLD + X to denote the result of using $\mathcal{D}_{\text{GOLD}}$ as the retrieval pool in the setting of X -shot inference.

1. The very large telescope was built in the early 1990s by the European Southern Observatory (ESO) in Chile.
The Very Large Telescope.
2. The Hubble Space Telescope.
3. The first pictures of a planet outside of our own solar system were taken by the Hubble Space Telescope in 1990.
4. The Hubble Space Telescope.
5. The very first pictures of a planet outside of our own solar system were taken by the Hubble Space Telescope.

The results of gpt2-medium (denoted as MED in the experiment) show that only the 4-st statement is not a sufficient plug-in instance:

1. The Very Large Telescope.
2. The Very Large Telescope.
3. The Very Large Telescope.
4. The Hubble Space Telescope.
5. The Very Large Telescope.

All the above results verify that *quality* of plug-in data should be LM-specific, which is the key idea of our paper.

A4.5 Complexity Study

We report the time complexity of running the sufficient tree search algorithm (i.e., Algorithm A4) on COLA and TREC datasets in Figure A1. From the figure, we can observe that as the number of

samples reduces, the time consumption of Algorithm A4 would also reduce. Combining Figure A1 and 4, we can see that the time consumption is almost linear to the size of data samples.

We also report the time complexity of inference time of NEO in the context of the unsupervised learning on COLA dataset in Figure A2, which includes running our necessary plug-in data miner. From the figure, we can see that our method using $\mathcal{D}_{\text{GOLD}}$ instead of $\mathcal{D}_{\text{TRAIN}}$ can largely reduce the computation cost. And, since we employ Algorithm A3 with $R_{\text{MAX}} = 1$ as our necessary plug-in data miner, it almost makes no difference in the time cost.